

Round-Efficient Composable Two-Party Quantum Computation

Vipul Goyal¹, Xiao Liang^{2*}, Omkant Pandey^{3†}, Yuhao Tang³, and Takashi Yamakawa⁴

¹ NTT Research & Carnegie Mellon University, USA
vipul@cmu.edu

² The Chinese University of Hong Kong, Hong Kong
xiaoliang@cuhk.edu.hk

³ Stony Brook University, USA
{omkant,yuhtang}@cs.stonybrook.edu

⁴ NTT Social Informatics Laboratories, Japan
takashi.yamakawa@ntt.com

Abstract. We study secure computation in the plain model against fully concurrent quantum adversaries. While classical simulation-based notions — such as Super-Polynomial Simulation (SPS) security — have enabled meaningful forms of concurrent security, very little is known about their quantum counterparts, particularly under standard polynomial-time hardness assumptions.

Our main result is the first post-quantum two-party computation protocol that achieves concurrent SPS security, based solely on the minimal assumption of semi-honest post-quantum oblivious transfer (PQ-OT). Moreover, our protocol has constant round complexity when the underlying PQ-OT protocol is constant-round. This can be viewed as a post-quantum analog of the classical result by Garg et al. [Eurocrypt’12], but with a crucial difference: our security proof completely avoids rewinding, making it suitable for quantum settings where rewinding is notoriously challenging due to the no-cloning principle.

By leveraging a compiler of Bartusek et al. [Crypto’21], we further extend our result to the fully quantum setting, yielding the first constant-round concurrent SPS two-party computation for *quantum* functionalities in the plain model.

Additionally, we construct a two-round, public-coin, concurrent SPS post-quantum zero-knowledge protocol for languages in $\mathbf{NP} \cap \mathbf{coNP}$, under the quantum polynomial-time hardness of LWE. This result is notable even in the classical setting.

Keywords: Two-Party Computation · Post-Quantum · Concurrency · Super-Polynomial Simulation · Zero-Knowledge

* The work described in this paper was partially supported by a grant from the Research Grants Council of the Hong Kong Special Administrative Region, China [Project No.: CUHK 24208025].

† Supported in part by AFOSR Award FA9550-24-1-0156, NSF CAREER Award 2144303, and NSF grant 2106263. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the United States Government, AFOSR, or NSF.

Table of Contents

Abstract	i
Table of Contents	ii
1 Introduction	1
1.1 Our Results	2
1.2 Related Work	3
2 Technical Overview	3
2.1 Concurrent SPS PQ-2PC	3
2.1.1 Starting Point: the [GGJS12] Protocol	3
2.1.2 Challenges in the Post-Quantum Setting	5
2.1.3 Our Approach	5
2.2 Concurrent SPS PQ-ZK for $\mathbf{NP} \cap \mathbf{coNP}$	7
3 Preliminaries	7
3.1 Basic Notations	7
3.2 Semi-Honest Two Party Computation	8
3.3 Post-Quantum Secure Function Evaluation	9
3.4 Collision Resistant Hash Function	9
3.5 Commitment Schemes	10
3.5.1 Non-Malleable Commitments	11
3.5.2 k-Robustness	11
3.5.3 First-Message Binding	12
3.6 Interactive Proofs and Arguments	12
4 Post-Quantum Constant-Round Concurrent 2PC	14
4.1 Proof of Security	15
5 Two-Round SPS-ZK for $\mathbf{NP} \cap \mathbf{coNP}$	21
5.1 Building Blocks	21
5.1.1 ZAPs for Controlled-OR Languages	21
5.2 Construction	22
5.3 Proof of Security	23
5.4 Proof of Soundness	24
5.5 Proof of Zero-Knowledge	24
6 Two-Round SPS-ZK for $\mathbf{NP}$	25
6.1 Non-interactive Super-Polynomial Time Extractable Commitment	25
6.1.1 Construction of Non-Interactive SPS-Extractable Commitment	25
6.2 Post-Quantum Two-Round Super-Polynomial Time Simulatable Zero-Knowledge	26
6.3 Proof of Security	26
6.4 Proof of Soundness	26
6.5 Proof of Zero-Knowledge	27
References	29

1 Introduction

Secure multi-party computation (MPC) [Yao86, GMW87] is a cryptographic protocol that enables multiple parties to jointly compute a function over their private inputs, while ensuring that no party learns anything about the others’ inputs beyond what can be inferred from the final output. Traditionally, MPC has been studied in the stand-alone setting, where only a single execution of the protocol is considered. However, in real-world applications, it is more realistic to assume that multiple protocol instances, possibly of different protocols, may be executed concurrently.

To model and ensure security in such concurrent environments, Canetti [Can01] introduced the framework of *Universally Composable* (UC) security. UC security provides strong guarantees: it ensures that a protocol remains secure even when composed with arbitrary other protocols running concurrently. Despite its desirable properties, achieving UC security is known to be impossible in the plain model (i.e., without trusted setup) for most non-trivial functionalities. This impossibility result holds even when restricted to concurrent executions of the *same* protocol (a setting known as *self-concurrency*) [Lin03, Lin04].

These strong negative results have led to the exploration of alternative notions of security that can still provide meaningful guarantees in concurrent settings. Notable (simulation-based) notions include: (1) *Super-Polynomial Simulation* (SPS) security [Pas03], a variant of the standard simulation-based security that permits the simulator to run in super-polynomial time, (2) *Angel-Based Simulation* [PS04] and its extension to (interactive) super-polynomial helpers [CLP10], where the simulator itself is polynomial time but has access to an super-polynomial-time oracle, (3) *Shielded-Oracle Model* [BDH⁺17] which offers a middle ground between the first two notions, and (4) *Multiple Ideal-Queries Model* [GJ13] where the simulator is polynomial time but can make multiple queries to the ideal functionality.

While early works in this direction required super-polynomial hardness assumptions [PS04, BS05, MMY06, LPV09], Canetti, Lin, and Pass [CLP10] proposed techniques to achieve angel-based simulation under (standard) polynomial-time assumptions. Later, similar results were also achieved for other variants such as SPS and Shielded-oracle [GGJS12, BDH⁺17, GKP18]. These concurrency models are extensively studied and offer a range of robust solutions with different trade-offs when standard polynomial-time simulation becomes unachievable.

Concurrent Security in the Post-Quantum Setting. The above positive results consider only classical security, and it remains unclear whether they hold against quantum adversaries. In fact, it has been observed that classical proofs of simulation-based security often do not directly extend to the post-quantum setting [Gra97, DFS04]. A central obstacle in the post-quantum setting is the no-cloning principle, which prevents duplication of the quantum adversary’s internal state and thus undermines classical *rewinding*-based proof techniques. It is worth noting that rewinding techniques are critical to achieve security under polynomial hardness **even when super-polynomial simulators are allowed**, e.g., in SPS and angel-based notions of security mentioned above.

These challenges have motivated a growing line of research, yielding exciting development of quantum simulation techniques. These include the construction of post-quantum zero-knowledge proofs (PQ-ZK) [Wat09, BS20, CCY21, LMS22], post-quantum secure two-party computation protocols (PQ-2PC) protocols [DL09, LN11, HSS11, CCLY22], and even post-quantum secure MPC protocols (PQ-MPC) [ABG⁺21, LPY23, CLPY25].

However, all of these results were obtained in the *stand-alone* setting. When it comes to concurrent quantum security, rather surprisingly, very little is known⁵. Two works [ACL21, GLM23] address the so-called *bounded-concurrent* setting, but this model is limited: it requires the protocol designer to know an a priori bound on the total number of concurrent sessions; if the number of sessions exceeds this bound in a real-world concurrent scenario, no security guarantees are provided.

It is also worth noting that some classical works achieve concurrent SPS 2PC under only the Learning with Errors (LWE) assumption and using only straight-line simulators (e.g., [ABF⁺23], or [BGI⁺17] combined with [BD18]). These constructions would naturally remain secure even against quantum adversaries. However, they rely on the *super-polynomial* hardness of LWE, and we know almost nothing under standard *polynomial*-

⁵ This work focuses on constructions without trusted setup assumptions. If such assumptions are allowed (e.g., a common reference string), there exist known classical UC-MPC protocols that remain secure in the quantum setting, provided they are based on quantum-secure building blocks and admit a straight-line simulator.

time hardness assumptions. This situation is quite unsatisfactory, especially when contrasted with the rich literature on classical concurrent security as described earlier.

As the primary goal of this work, we aim to improve this unsatisfactory state of affairs by developing techniques that can effectively address (fully) concurrent attacks by a quantum adversary. More specifically, we focus on the following fundamental question:

Question: *Does there exist setup-free PQ-2PC from standard **polynomial hardness** assumptions in any meaningful concurrent model (and ideally in a small number of rounds)?*

Note that we aim for a *fully* concurrent model here, in contrast to the bounded concurrency setting studied in prior works [ACL21, GLM23].

1.1 Our Results

In this work, we affirmatively answer the above question. We also obtain additional positive results for the case of PQ-ZK.

Post-Quantum SPS-2PC. Our first result is a construction of constant-round concurrently SPS-secure PQ-2PC (PQ-SPS-2PC) *under the minimal assumption* that post-quantum oblivious transfers (PQ-OTs) exists:

Theorem 1. *Assuming the existence of constant-round semi-honest PQ-OTs, there exists a constant-round concurrent SPS post-quantum two-party computation.*

Our result can be seen as a post-quantum analog of the result of [GGJS12]. Our main technical innovation lies in developing a new proof of security that, roughly speaking, avoids the use of any rewinding⁶. The use of rewinding was critical in [GGJS12] in order to achieve a construction under polynomial hardness. Thus, in avoiding the use of any rewinding, our proof departs significantly from that of [GGJS12]. Establishing the concurrent post-quantum security of our protocol turns out to be highly nontrivial.

Post-Quantum SPS-ZK. Our second result is concerned with the construction of concurrent PQ-ZK with super-polynomial simulation (PQ-SPS-ZK). While PQ-SPS-ZK is a special case of PQ-SPS-2PC, and thus follows from it, our focus is on a (tighter) two-round construction. A two-round construction of PQ-SPS-ZK under quantum *subexponential* hardness of LWE already follows as a corollary of the results in [ABF⁺23]. (In Sec. 6, we also present a direct construction that additionally satisfies the public-coin property.) In contrast, almost nothing is known for this object under **polynomial** hardness assumption.

We show that two-round PQ-SPS-ZK can be achieved under quantum **polynomial** hardness of LWE if we focus on languages in $\mathbf{NP} \cap \mathbf{coNP}$:

Theorem 2. *Assuming the quantum polynomial-time hardness of LWE, there exist two-round, public-coin, concurrent PQ-SPS-ZK for $\mathbf{NP} \cap \mathbf{coNP}$.*

We remark that such a result was not known even in the classical setting (even without requiring the public-coin property).

Extension to Quantum Functionalities. The discussion so far has focused on the post-quantum setting, where honest parties and their communication channels remain classical. In the quantum regime, however, it is also desirable to grant honest parties quantum computational and communication capabilities, enabling them to securely compute quantum functionalities (e.g., preparing entanglement or performing quantum key distribution).

Indeed, this is a compelling direction that has attracted significant research attention [DL09, LN11, HSS11, DNS12, DGJ⁺20, ACC⁺21, GLSV21, BCKM21b, BCKM21a, LPY23]. However, this line of work focuses on the stand-alone setting, and none of these results address the concurrent setting in the plain model (i.e., without trusted setup assumptions). The closest attempts are again the works of [ACL21] and

⁶ Technically speaking, our reduction relies on secure primitives such as non-malleable commitments and zero-knowledge proofs, and the security proofs for those primitives may use rewinding. However, our reduction does not introduce rewinding of its own.

[GLM23], whose respective limitations have been discussed earlier. *Thus, almost nothing is known about the (unbounded) concurrency of plain-model secure quantum computation!*

We observe that our [Thm. 1](#) can be extended to the fully quantum setting using the compiler developed in [\[BCKM21a\]](#). In particular, [\[BCKM21a, Theorem 5.1\]](#) presents a compiler that transforms any post-quantum secure two-party computation (PQ-2PC) protocol into a secure two-party *quantum* computation (2PQC) protocol (i.e., for computing quantum functionalities), incurring only a constant number of additional rounds. Moreover, the security proof of this compiler is “straight-line,” in the sense that the simulator for the resulting 2PQC protocol does not rely on rewinding techniques *when viewed in the PQ-2PC hybrid model*. Thanks to this feature, we observe the following: as long as the underlying PQ-2PC protocol is concurrently secure and admits a straight-line SPS simulator, the 2PQC protocol produced via the [\[BCKM21a\]](#) compiler will also be concurrently secure with a straight-line SPS simulator. This observation, combined with [Thm. 1](#), yields the following corollary:

Corollary 1. *Assuming the existence of constant-round semi-honest PQ-OTs, there exists a constant-round concurrent SPS two-party quantum computation.*

1.2 Related Work

In the classical setting, constant-round concurrent SPS-MPC (as opposed to 2PC) has been constructed under standard polynomial-time assumptions in [\[GKP18\]](#) (building upon [\[LP12, Kiy14, GLP⁺15\]](#)). Notably, their construction relies only on black-box use of cryptographic primitives, in contrast to ours (which follows the approach of [\[GGJS12\]](#)). An important open question is whether a post-quantum analog of [\[GKP18\]](#) can be achieved, that is, whether it is possible to construct a constant-round concurrent PQ-SPS-MPC protocol based on a polynomial-time assumption, ideally using a black-box construction.

We note that Angel-based UC [\[PS04, CLP10\]](#) and Shielded Oracle UC [\[BDH⁺17\]](#) are closely related generalizations of SPS-security. Unlike concurrent SPS security, these notions have the advantage of supporting a universal composition theorem. However, they often require either more rounds or somewhat stronger assumptions than SPS-security. The state-of-the-art protocols under these security notions include an $\tilde{O}(\log \lambda)$ -round black-box construction based on constant-round semi-honest OT for Angel-based UC [\[CLP20\]](#), and a constant-round black-box construction under stronger polynomial-time assumptions for Shielded Oracle UC [\[BDH⁺17\]](#).

2 Technical Overview

2.1 Concurrent SPS PQ-2PC

2.1.1 Starting Point: the [\[GGJS12\]](#) Protocol

Before presenting our post-quantum, constant-round, concurrently secure two-party computation (2PC) protocol, we first review the *classical-secure* constant-round concurrent 2PC protocol introduced by [\[GGJS12\]](#). It is worth mentioning that, prior to [\[GGJS12\]](#), all known constructions of secure computation protocols achieving SPS security either rely critically on a super-polynomial time simulator to break some underlying cryptographic primitives and extract the trapdoor or, in an effort to avoid complexity-leveraging, have to deal with *concurrent rewinding* in some forms, which is often challenging and leads to a substantial increase in round complexity.

To bypass the need for concurrent rewinding, the key idea of [\[GGJS12\]](#) is to ensure that the simulator $\mathcal{S}$ can always extract the trapdoor before it begins to “cheat” for the rest of the protocol session, even in the complex setting where multiple instances of the protocol is running concurrently, and proceeds with a straight-line simulation for the rest of the protocol session. More concretely, [\[GGJS12\]](#) employs a non-malleable zero-knowledge argument (NMZK) scheme to set up a trapdoor, which enables safe extraction even in the concurrent setting.⁷ Following this, the protocol proceeds with a semi-honest two-party computation,

⁷ We remark that a special NMZK scheme is used here, which can be divided into a *preamble phase* and a *post-preamble phase*. Roughly speaking, the preamble phase consists of an extractable commitment to some “trapdoor”, while the post-preamble phase is where the actual proof takes place.

where each party proves—at every step—that it is behaving honestly, using a witness-indistinguishable (WI) proof.

A high-level overview of their construction is provided in [Prot. 1](#).

Protocol 1: Classical-Secure Constant-Round Concurrent 2PC
Phase I: Trapdoor Generation^a 1. (Preamble Phase of NMZK) P_A “sends” an extractable commitment (ExtCom) of a random string σ to P_B; P_B acts symmetrically. 2. (Trapdoor) P_A sends com_1, a commitment of bit 0 to P_B. 3. (Post-preamble Phase of NMZK) The two parties execute the “post-preamble phase” of NMZK, where P_A proves to P_B that com_1 is a commitment to bit 0. 4. P_B acts symmetrically. Phase II: Input Commitment^b In this phase, the parties commit to their inputs and random coins (to be used in the next phase) using ExtCom, and each party attaches a witness-indistinguishable (WI) proof showing that it either “knows” the trapdoor or has sent a valid extractable commitment. Phase III: Secure Computation In this phase, P_A and P_B engage in an execution of the semi-honest two-party protocol, and each party attaches a WI proof after every semi-honest 2PC message, proving that it either “knows” the trapdoor or behaves honestly “so far” in the protocol. ^a The purpose of Phase I is to aid the simulator $\mathcal{S}$ in obtaining a “trapdoor” to be used during the simulation of the protocol. ^b The purpose of Phase II is to aid the $\mathcal{S}$ in extracting the adversary’s input and randomness.

Before delving into the security analysis of [\[GGJS12\]](#), we first briefly recall a canonical classically-secure extractable commitment scheme ExtCom (e.g., [\[PW09, GGJS12\]](#)). The first message of ExtCom statistically binds the committed value, and extraction is achieved by rewinding the committer multiple times with different second-message challenges and collecting the corresponding responses. The committed value can then be recovered using these transcripts. Additionally, each such execution—consisting of a second message and its corresponding response—is referred to as a look-ahead thread.

In the security proof of [\[GGJS12\]](#), the simulator $\mathcal{S}$ performs rewinding only within dedicated look-ahead threads to extract the trapdoor from ExtCom, and proceeds with a straight-line simulation in the main thread. Crucially, within look-ahead threads, [\[GGJS12\]](#) allows $\mathcal{S}$ to use *honest parties’ secret inputs* (in intermediate hybrid experiments). More specifically, if $\mathcal{S}$ initiates multiple look-ahead threads to extract the trapdoor committed in the ExtCom of protocol session j , then for each session $i \neq j$, it proceeds as follows: if the trapdoor for session i is available, $\mathcal{S}$ performs straight-line simulation for session i within the look-ahead threads; otherwise, $\mathcal{S}$ uses the honest party’s input for session i within look-ahead threads. Additionally, in their security proof, [\[GGJS12\]](#) introduced a novel approach for organizing the sequence of hybrid experiments. The conventional approach organizes hybrids in a “session-by-session” manner, wherein all modifications to a given session i are completed consecutively (through a sequence of successive hybrids) before proceeding to any other session, independent of the adversary’s scheduling. In contrast, [\[GGJS12\]](#) designed a sequence of hybrids that interleave across multiple sessions, with transitions “jumping” between sessions as dictated by the adversary’s message scheduling. As a result, the exact order of hybrids depends dynamically on the adversary’s scheduling decisions. By leveraging the above techniques, [\[GGJS12\]](#) avoids dealing with complex concurrent rewinding used in previous works and instead relies solely on “local rewindings” within look-ahead threads. As a result, their protocol achieves constant-round while relying only on polynomial-time assumptions.

2.1.2 Challenges in the Post-Quantum Setting

However, achieving post-quantum security presents several challenges. First, recall that in [GGJS12], the simulator $\mathcal{S}$ must rewind the adversary $\mathcal{A}$ in “look-ahead threads” in order to extract the trapdoor—or, alternatively, $\mathcal{A}$ ’s input and randomness. This technique is generally incompatible with the quantum setting: due to the no-cloning theorem, $\mathcal{S}$ cannot record $\mathcal{A}$ ’s internal quantum state between look-ahead threads, and performing intermediate measurements would also disturb $\mathcal{A}$ ’s internal quantum state. As a result, even if extraction succeeds, $\mathcal{S}$ may not be able to faithfully simulate $\mathcal{A}$ ’s output state, breaking the simulation. In recent years, there has been significant progress in quantum rewinding and simulatable-extraction techniques [Wat09, CCY21, LMS22, CCLY22]. This naturally raises the question: can these new quantum rewinding techniques be combined with the structured rewinding order used in the original proof of [GGJS12] to lift their security analysis to the quantum setting?

Upon closer examination, this approach appears to be quite challenging—if not fundamentally impossible. There are new obstacles that have not been addressed in the existing quantum rewinding literature. In particular, these techniques are not applicable in the concurrent setting due to the adversary’s ability to arbitrarily schedule messages. We elaborate on this in the following discussion.

In [GGJS12], after the simulator $\mathcal{S}$ initiates multiple look-ahead threads to extract the trapdoor (or the adversary $\mathcal{A}$ ’s input), it must return to the execution of the main thread. More precisely, $\mathcal{S}$ must first “enter” a look-ahead thread from a specific “entering-point” in the concurrent schedule—namely, the second message of `ExtCom`, and return to the “entering-point” to begin its execution of the main thread. As we mentioned earlier, to perform extraction, $\mathcal{S}$ sends a different second-message challenge, records the corresponding response, and then rewinds $\mathcal{A}$ to *return to the same “entering-point”*. This process is repeated multiple times to gather sufficient transcripts for extraction.

While this is an easy task in the classical setting, in sharp contrast, this becomes significantly more challenging in the quantum setting — rather surprisingly, no quantum rewinding techniques in existing literature are known to achieve this effect! Indeed, this task seems impossible as we explain below: a direct quantum analog of this requirement would, at the very least, demand the ability to estimate $\mathcal{A}$ ’s probability of aborting while simultaneously simulating $\mathcal{A}$ ’s internal state at the entering point—that is, prior to initiating the rewinding procedure. This seems inherently infeasible: for example, if $\mathcal{A}$ ’s internal state at the “entering-point” is an equal superposition of an honest committer and an always-aborting adversary, then there is no hope to recover $\mathcal{A}$ ’s original initial state after we collapse its “aborting or not” decision.

There is another, less obvious obstacle, which is more subtle and harder to describe at this point. We defer a detailed discussion to the next subsection, where it will arise naturally in the context of our main ideas.

2.1.3 Our Approach

In this work, we aim to construct post-quantum constant-round concurrent secure two-party computation under the minimal assumption that constant-round oblivious transfer (OT) exists. Rather than tackling the seemingly harder problem of building post-quantum simulatable-extraction that remains robust against arbitrary message scheduling, we make a key observation: rewinding is, in fact, entirely unnecessary and can be removed from all intermediate hybrids. For simplicity, we refer to the $2m$ occurrences of `ExtCom` across the m sessions as *special commitments* throughout the remainder of this technical overview. In [GGJS12], roughly speaking, the authors considered a numbered ordering over all $2m$ special commitments⁸ across m sessions (2 special commitments for each session), and associated each occurrence of special commitment with a corresponding hybrid. For every $i \in [2m]$, the hybrid H_i (associated with the i th special commitment) will start multiple look-ahead threads to extract the trapdoor (or $\mathcal{A}$ ’s input) committed in the i th special commitment. A key structural property of the hybrids is that any future hybrids H_j (for $j > i$) will never make any modifications to the look-ahead threads initiated by H_i . Moreover, in H_i , no look-ahead threads are initiated after the i th special commitment has been sent. That is, in hybrid H_i , all values committed in the first i special commitment are extracted via look-ahead threads. Since the final simulator $\mathcal{S}$ does not perform any rewindings and instead runs in super-polynomial time to extract trapdoors (or $\mathcal{A}$ ’s input and

⁸ In [GGJS12], the authors actually considered $4m$ special *messages*; we simplify this to $2m$ special commitments for clarity.

randomness), we are likewise allowed to use super-polynomial power to extract trapdoors (or $\mathcal{A}$'s input and randomness) from the first i_{th} special commitment in H_i , for any $i \in [2m]$, provided that we can demonstrate indistinguishability between the hybrids. This crucial observation allows us to avoid the need for any quantum rewinding or simulatable extraction. Consequently, we can significantly weaken the requirements on the underlying commitment scheme and replace each occurrence of `ExtCom` with a non-interactive, statistical binding commitment⁹.

Additionally, since H_i extracts trapdoors using super-polynomial power instead of relying on look-ahead threads, we eliminate the need for a post-quantum, constant-round, public-coin non-malleable commitment `NMCom` scheme (which serves as a building block in the `NMZK` construction). Notably, the existence of such a primitive in the post-quantum setting remains an open problem.

In hybrid H_i , once $\mathcal{S}$ extracts the trapdoor for session i , it begins to “cheat” in the remainder of session i . However, we must argue that, when $\mathcal{S}$ begins to “cheat” in session i (viewed as a left interaction), the adversary $\mathcal{A}$ cannot “cheat” simultaneously in any other session (viewed as right interactions). Intuitively, this can be reduced to the non-malleability of `NMCom`. Nevertheless, several subtleties arise in making this reduction precise.

- The standard security guarantee of non-malleability holds only in settings where a man-in-the-middle adversary $\mathcal{M}$ participates in two executions of the *same* protocol. Consequently, when H_i begins to “cheat” using the extracted trapdoor in session i , we cannot directly invoke non-malleability to argue that to argue that the adversary $\mathcal{A}$ cannot simultaneously cheat in other sessions. To address this, [GGJS12] relies on the statistical security of the underlying primitives. In our work, we resolve this issue by leveraging an additional property of `NMCom` known as *robustness*, first introduced by [LP09] and has recently become a key ingredient in the design of post-quantum non-malleable commitments [BLS22, LPY23, CLPY25]. Robustness ensures that non-malleability holds regardless of which protocol is used in the left interaction, allowing us to rule out any cheating behavior by $\mathcal{A}$ in other sessions—even when $\mathcal{S}$ is “cheating” in session i .
- To reduce $\mathcal{A}$'s cheating behavior to the (robust) non-malleability of `NMCom`, we need to construct a man-in-the-middle adversary $\mathcal{M}$ that runs $\mathcal{A}$ internally and interacts with an external committer C and a receiver R . However, in [GGJS12], the hybrid H_i initiates multiple look-ahead threads to extract trapdoor from session i . Due to $\mathcal{A}$'s arbitrary message scheduling, it is possible that in some session $j \neq i$, the first message of `NMCom` of session j is sent *before* the i_{th} special commitment. In such cases, $\mathcal{S}$ would need to rewind the external receiver R in the look-ahead threads. This presents a problem, as R is a stand-alone external party and only responds in a single thread of execution. [GGJS12] resolves this issue by leveraging the *public-coin* property of `NMCom`. In their approach, all `NMCom` messages on the main-thread are forwarded between $\mathcal{A}$ and the external receiver R ; On the other hand, in look-ahead threads, any message that would have been sent by R to $\mathcal{A}$ is instead replaced with a random string by $\mathcal{M}$, and this is valid only because `NMCom` is a public-coin protocol.

In contrast, our protocol eliminates rewinding entirely, thereby removing the need for look-ahead threads. As a result, we no longer require `NMCom` to be public-coin. However, this leads to a new issue: in our new approach, H_i uses super-polynomial power to extract trapdoors (or $\mathcal{A}$'s input and randomness) from the first i special commitment. Similarly, if there exists a session $j \neq i$ such that the first message of `NMCom` of session j is sent before the i_{th} special commitment, then we cannot reduce this to the non-malleability of `NMCom`, simply because we are still using super-polynomial power during the execution of `NMCom`. We resolve this issue by requiring an additional property of `NMCom` called *first-message binding*, which requires the first message of `NMCom` should be always sent by the committer C and additionally binds the value of `NMCom`¹⁰. Thus, for session $j \neq i$, if `NMCom` of session j starts before the i_{th} special commitment, the value of `NMCom` is already bound and thus independent of $\mathcal{S}$'s “cheating behavior.”

For the detailed construction and security proof of our protocol, please refer to [Sec. 4](#).

⁹ Similar to [GGJS12], the non-interactive property is not strictly necessary for our construction, but is chosen for simplicity of presentation.

¹⁰ For simplicity and clarity, we assume that `NMCom` begins with a non-interactive, statistically binding commitment. However, the non-interactive property can be relaxed by having both parties exchange the first message of Naor's commitment at the beginning of the protocol. This allows the commitment used within `NMCom` to be treated as effectively non-interactive in the remainder of the protocol.

2.2 Concurrent SPS PQ-ZK for $\mathbf{NP} \cap \mathbf{coNP}$

Although a post-quantum two-round SPS-ZK protocol can be obtained via a variant of the construction proposed by [Pas03] (please refer to Sec. 6), this approach inherently relies on super-polynomial assumptions.

Our construction relies on a collision resistant hash function (against QPT adversary) and a post-quantum function-hiding (also referred as circuit-private) secure function evaluation (SFE). The notion of circuit privacy requires that for any two functionally-equivalent circuits C_0 and C_1 of the same size, and any $\text{ct}^* \in \{0,1\}^*$, no QPT adversary can distinguish between $\text{Eval}(C_0, \text{ct}^*)$ and $\text{Eval}(C_1, \text{ct}^*)$. We provide a high-level description of our protocol in the following:

- $V \rightarrow P$: V sends a hash key s for CRHF, an SFE encryption ct of 0, and the first message of ZAP to P .
- $P \rightarrow V$: The prover P begins by computing $\hat{\text{ct}} \leftarrow \text{Eval}(C^0, \text{ct})$, where C^0 is a constant-0 circuit that always outputs 0s. Then, P sends the second message of ZAP, proving that either $x \in \mathcal{L}$, or that it “knows” a collision for the collision-resistant hash function and that $\hat{\text{ct}} = \text{Eval}(C', \text{ct})$, where C' is a circuit that checks whether the input is a no-witness for $\mathcal{L}$, and if so, outputs the collision; otherwise, it outputs 0.

The round complexity is clear from the description above. Moreover, as long as the underlying SFE protocol is public-coin, our construction remains public-coin as well. The soundness property follows from a standard reduction to collision-resistance. The proof of the super-polynomial-time simulatable zero-knowledge property is provided below.

Proof (Sketch). The simulator $\mathcal{S}$ begins by using brute force to find a collision for the collision-resistant hash function. Such a collision is guaranteed to exist due to the compressing property of the CRHF. Then, $\mathcal{S}$ computes $\hat{\text{ct}}$ as $\text{Eval}(C', \text{ct})$. Given that $x \in \mathcal{L}$, there exists no no-witness for $\mathcal{L}$, and thus C' and C are functionally equivalent. By the circuit privacy of SFE, $\hat{\text{ct}}$ is statistically indistinguishable from that in the real execution. Finally, $\mathcal{S}$ computes the second message of ZAP using the fact that $\hat{\text{ct}} = \text{Eval}(C', \text{ct})$ and it “knows” a collision as the witness. By the witness indistinguishability (WI) property of the ZAP, the simulated proof is indistinguishable from a real one. $\square$

Note that for the second message of ZAP, the prover P must prove $(x, \hat{\text{ct}}) \in \mathcal{L} \vee \mathcal{L}_{\text{ct}}$, where $\hat{\text{ct}} \in \mathcal{L}_{\text{ct}}$ if and only if $\hat{\text{ct}} = \text{Eval}(C', \text{ct})$ and it “knows” a collision. At first glance, this seems to require a ZAP for general $\mathbf{NP}$, since $\mathcal{L}_{\text{ct}} \notin \mathbf{NP} \cap \mathbf{coNP}$. However, by exploiting the fact that $x \in \mathbf{NP} \cap \mathbf{coNP}$, we observe that it suffices to use a ZAP with certain structural properties tailored to this setting.

- If $x \in \mathcal{L}$, we place no requirement on the soundness of the ZAP. Otherwise, we require that no valid second message zap_2 exists unless $\hat{\text{ct}} \in \mathcal{L}_{\text{ct}}$.
- If $x \in \mathcal{L}$, we require the ZAP to satisfy witness indistinguishability. Otherwise, no such requirement is imposed.

We refer to ZAPs with the above properties as *ZAPs for Controlled-OR Languages*, and provide a formal definition in Def. 19. Such ZAPs can be instantiated via the construction given in [CGH⁺21, CCLM22]¹¹, under the polynomial-time hardness of LWE.

For the detailed construction and security proof of our protocol, please refer to Sec. 5.

3 Preliminaries

3.1 Basic Notations

Let $\lambda \in \mathbb{N}$ denote security parameter. For a positive integer n , let $[n]$ denote the set $\{1, 2, \dots, n\}$. For a finite set $\mathcal{X}$, $x \leftarrow \mathcal{X}$ means that x is uniformly chosen from $\mathcal{X}$.

A function $f : \mathbb{N} \rightarrow [0, 1]$ is said to be *negligible* if for all polynomial p and sufficiently large $\lambda \in \mathbb{N}$, we have $f(\lambda) < 1/p(\lambda)$; it is said to be *overwhelming* if $1 - f$ is negligible, and said to be *noticeable* if there

¹¹ In [CGH⁺21, CCLM22], the authors define and construct ZAPs for super-complement languages. While their definition does not directly imply our notion of ZAPs for Controlled-OR Languages, we emphasize that their construction satisfies our definition, and the corresponding security proof remains essentially unchanged.

is a polynomial p such that $f(\lambda) \geq 1/p(\lambda)$ for sufficiently large $\lambda \in \mathbb{N}$. We denote by **poly** an unspecified polynomial and by **negl** an unspecified negligible function.

Honest (classical) parties are modeled as interactive Turing machines (ITMs). We use PPT and QPT to denote (classical) probabilistic polynomial time and quantum polynomial time, respectively. For a classical probabilistic or quantum algorithm $\mathcal{A}$, $y \leftarrow \mathcal{A}(x)$ means that $\mathcal{A}$ is run on input x and outputs y . An adversary (or malicious party) is modeled as a non-uniform PPT (or QPT in the case of post-quantum security) algorithm. When we consider a non-uniform QPT adversary, we often specify it by a sequence of polynomial-size quantum circuits with quantum advice $\{\mathcal{A}_\lambda, \rho_\lambda\}_{\lambda \in \mathbb{N}}$. In an execution with the security parameter λ , $\mathcal{A}$ runs $\mathcal{A}_\lambda$ taking ρ_λ as the advice. For simplicity, we often omit the index λ and just write $\mathcal{A}(\rho)$ to mean a non-uniform QPT algorithm specified by $\{\mathcal{A}_\lambda, \rho_\lambda\}_{\lambda \in \mathbb{N}}$.

For an **NP** language $\mathcal{L}$ and a true statement in this language $x \in \mathcal{L}$, we use $\mathcal{R}_\mathcal{L}(x)$ ($\mathcal{R}$ stands for “relation”) to denote the set of all witnesses for x . We will refer to the OR-composition of **NP** languages, which are defined in [Def. 1](#). The following definitions are based on those in [\[LPY23\]](#)

Definition 1 (OR-Composition of NP Languages). *Let $\mathcal{L}_1$ and $\mathcal{L}_2$ be two NP languages. The OR-composition of them (dubbed $\mathcal{L}_1 \vee \mathcal{L}_2$) is the new NP language defined as follows:*

$$\mathcal{L}_1 \vee \mathcal{L}_2 := \{(x_1, x_2) \mid x_1 \in \mathcal{L}_1 \vee x_2 \in \mathcal{L}_2\}.$$

Notations for Indistinguishability. We may consider random variables over bit strings or over quantum states. This will be clear from the context. We use the same notations for classical and quantum computational indistinguishability, but there should be no fear of confusion; It means computational indistinguishability against PPT (resp. QPT) distinguishers whenever we consider classical (resp. post-quantum) security. For ensembles of random variables $\mathcal{X} = \{X_i\}_{\lambda \in \mathbb{N}, i \in I_\lambda}$ and $\mathcal{Y} = \{Y_i\}_{\lambda \in \mathbb{N}, i \in I_\lambda}$ over the same set of indices $I = \bigcup_{\lambda \in \mathbb{N}} I_\lambda$ and a function δ , we use $\mathcal{X} \stackrel{c}{\approx}_\delta \mathcal{Y}$ to mean that for any *non-uniform* PPT (resp. QPT) algorithm $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, $i \in I_\lambda$, we have

$$|\Pr[\mathcal{A}(X_i)] - \Pr[\mathcal{A}(Y_i)]| \leq \delta(\lambda) + \text{negl}(\lambda).$$

We say that $\mathcal{X}$ and $\mathcal{Y}$ are δ -computationally indistinguishable if the above holds. In particular, when the above holds for $\delta = 0$, we say that $\mathcal{X}$ and $\mathcal{Y}$ are computationally indistinguishable, and simply write $\mathcal{X} \stackrel{c}{\approx} \mathcal{Y}$.

Similarly, we use $\mathcal{X} \stackrel{s}{\approx}_\delta \mathcal{Y}$ to mean that for any unbounded time algorithm $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, $i \in I_\lambda$, we have

$$|\Pr[\mathcal{A}(X_i)] - \Pr[\mathcal{A}(Y_i)]| \leq \delta(\lambda) + \text{negl}(\lambda).$$

In particular, when the above hold for $\delta = 0$, we say that $\mathcal{X}$ and $\mathcal{Y}$ are statistically indistinguishable, and simply write $\mathcal{X} \stackrel{s}{\approx} \mathcal{Y}$. Moreover, we write $\mathcal{X} \equiv \mathcal{Y}$ to mean that X_i and Y_i are distributed identically for all $i \in I$.

When we consider an ensemble $\mathcal{X}$ that is only indexed by λ (i.e., $I_\lambda = \{\lambda\}$), we write $\mathcal{X} = \{X_\lambda\}_\lambda$ for simplicity.

3.2 Semi-Honest Two Party Computation

Definition 2 (Post-Quantum Semi-Honest Two Party Computation). *Let $\mathcal{F} = (f_A, f_B)$ be a PPT functionality : $\{0, 1\}^{\ell_{in}^A} \times \{0, 1\}^{\ell_{in}^B} \rightarrow \{0, 1\}^{\ell_{out}^A} \times \{0, 1\}^{\ell_{out}^B}$. We say that $\langle P_A^{\text{sh}}, P_B^{\text{sh}} \rangle$ securely computes $\mathcal{F}$ in the presence of static semi-honest adversaries if there exist QPT algorithms $\mathcal{S}_A$ and $\mathcal{S}_B$ such that for all $x_A \in \{0, 1\}^{\ell_{in}^A}$, $x_B \in \{0, 1\}^{\ell_{in}^B}$, and $\lambda \in \mathbb{N}$, it holds that:*

$$\begin{aligned} \{(\mathcal{S}_A(1^\lambda, x_A, \mathcal{F}_A(x_A, x_B))), \mathcal{F}(x_A, x_B)\}_{\lambda, x_A, x_B} &\stackrel{c}{\approx} \{((\text{view}_A(x_A, x_B, \lambda)), \text{out}(x_A, x_B, \lambda))\}_{\lambda, x_A, x_B} \\ \{(\mathcal{S}_B(1^\lambda, x_B, \mathcal{F}_B(x_A, x_B))), \mathcal{F}(x_A, x_B)\}_{\lambda, x_A, x_B} &\stackrel{c}{\approx} \{((\text{view}_B(x_A, x_B, \lambda)), \text{out}(x_A, x_B, \lambda))\}_{\lambda, x_A, x_B} \end{aligned}$$

where

- $\text{view}_A(x_A, x_B, \lambda) = (x_A, r_A, m_1, \dots, m_t)$ with r_A as the random coins used by a semi-honest A and $m_1, \dots, m_t$ as the messages from B to A . $\text{view}_B(x_A, x_B, \lambda)$ is defined analogously, with $m_1, \dots, m_t$ as the messages from A to a semi-honest B .
- $\text{out}(x_A, x_B, \lambda) = (\text{out}_A(x_A, x_B, \lambda), \text{out}_B(x_A, x_B, \lambda))$ is the joint output of both parties in an execution of $\langle P_A^{\text{sh}}, P_B^{\text{sh}} \rangle$ on inputs x_A, x_B with security parameter λ .

Instantiation. Post-quantum semi-honest two party computation satisfying [Def. 2](#) are known from semi-honest post-quantum oblivious transfer (PQ-OT).

3.3 Post-Quantum Secure Function Evaluation

We recall the definition of secure function evaluation protocols with statistical circuit privacy and post-quantum input privacy (e.g., defined in [\[BS20\]](#)).

Definition 3 (Post-Quantum 2-Message Function-Hiding SFE). A post-quantum two-message function hiding secure function evaluation (SFE) protocol SFE consists of four algorithms (Gen, Enc, Eval, Dec). They satisfy the following syntax:

- $k \leftarrow \text{Gen}(1^\lambda)$: a PPT algorithm that takes as input a security parameter 1^λ , and outputs a secret key k ;
- $\text{ct} \leftarrow \text{Enc}(k, x)$: a PPT algorithm that takes as input a key k a string x , and outputs a ciphertext ct ;
- $\hat{\text{ct}} \leftarrow \text{Eval}(C, \text{ct})$: a PPT algorithm that takes as input a classical polynomial-size circuit C and ciphertext ct , and outputs an evaluated ciphertext $\hat{\text{ct}}$;
- $\hat{x} = \text{Dec}(k, \hat{\text{ct}})$: a deterministic algorithm that takes as input a ciphertext $\hat{\text{ct}}$, and outputs a string $\hat{x}$.

The scheme satisfies the following properties:

1. **Correctness:** For all polynomial $s(\cdot)$, all $\lambda \in \mathbb{N}$, all size- $s(\lambda)$ circuit C , and input x for C , it holds that

$$\Pr[\text{Dec}(k, \hat{\text{ct}}) = C(x) : k \leftarrow \text{Gen}(1^\lambda); \text{ct} \leftarrow \text{Enc}(k, x); \hat{\text{ct}} \leftarrow \text{Eval}(C, \text{ct})] = 1.$$

2. **Post-Quantum Input Privacy:** For any polynomial $\ell(\cdot)$,

$$\{\text{ct} : k \leftarrow \text{Gen}(1^\lambda); \text{ct} \leftarrow \text{Enc}(k, x_0)\}_{\lambda, x_0, x_1} \stackrel{c}{\approx} \{\text{ct} : k \leftarrow \text{Gen}(1^\lambda); \text{ct} \leftarrow \text{Enc}(k, x_1)\}_{\lambda, x_0, x_1}$$

where $\lambda \in \mathbb{N}$ and $x_0, x_1 \in \{0, 1\}^{\ell(\lambda)}$.

3. **Statistical Circuit Privacy:** For all polynomial $s(\cdot)$,

$$\{\text{Eval}(C_0, \text{ct}^*)\}_{\lambda, C_0, C_1, \text{ct}^*} \stackrel{s}{\approx} \{\text{Eval}(C_1, \text{ct}^*)\}_{\lambda, C_0, C_1, \text{ct}^*},$$

where $\lambda \in \mathbb{N}$, C_0 and C_1 are two size- $s(\lambda)$, functionally-equivalent circuits, and $\text{ct}^* \in \{0, 1\}^*$.

Instantiation. Secure function evaluation schemes satisfying [Def. 3](#) are known from the quantum hardness of LWE [\[OPP14, BD18\]](#).

3.4 Collision Resistant Hash Function

We recall the definition of post-quantum secure collision resistant hash function below.

Definition 4 (Post-Quantum Secure Collision Resistant Hash Function). A post-quantum secure collision resistant hash function (CRHF) is a pair of probabilistic polynomial-time algorithms (Gen, H) satisfying the following syntax:

- $s \leftarrow \text{Gen}(1^\lambda)$: a PPT algorithm that takes as input a security parameter 1^λ and outputs a key s . We assume that 1^λ is implicit in s .
- $y \leftarrow H^s(x)$: a deterministic polynomial algorithm that takes as input a key s and a string $x \in \{0, 1\}^{\ell^{\text{in}}(\lambda)}$ and outputs a string $y \in \{0, 1\}^{\ell^{\text{out}}(\lambda)}$ (where λ is the value of the security parameter implicit in s).

The scheme satisfies the following properties:

1. **Compressing:** $\ell^{\text{in}}(\lambda) > \ell^{\text{out}}(\lambda)$.
2. **Collision-resistant:** For every non-uniform QPT adversary $\{\mathcal{A}_\lambda, \rho_\lambda\}_{\lambda \in \mathbb{N}}$,

$$\Pr[(x_1 \neq x_2) \wedge (h^s(x_1) = h^s(x_2)) : (x_1, x_2) \leftarrow \mathcal{A}_\lambda(s, \rho_\lambda); s \leftarrow \text{Gen}(1^\lambda)] \leq \text{negl}(\lambda).$$

We recall the definition of post-quantum sub-exponentially secure collision-resistant hash function below.

Definition 5 (Post-Quantum Sub-Exponentially Secure Collision-Resistant Hash Function).

A post-quantum sub-exponential secure collision resistant hash function is a CRHF that additionally satisfies the following:

- **Sub-Exponential Collision-Resistant:** If there exist a κ , such that for every non-uniform probabilistic quantum adversary with circuit size bounded by $2^{\ell^{\text{out}}(\lambda)^\kappa}$ and takes a polynomial size quantum advice $\{\mathcal{A}_\lambda, \rho_\lambda\}_{\lambda \in \mathbb{N}}$,

$$\Pr[(x_1 \neq x_2) \wedge (h^s(x_1) = h^s(x_2)) : (x_1, x_2) \leftarrow \mathcal{A}_\lambda(s, \rho_\lambda); s \leftarrow \text{Gen}(1^\lambda)] \leq 2^{-\ell^{\text{out}}(\lambda)^\kappa}.$$

Instantiation. It is known that post-quantum sub-exponentially secure collision-resistant hash function satisfying Def. 5 can be constructed under the quantum hardness of LWE.

3.5 Commitment Schemes

We define post-quantum secure commitments. The following definitions are taken verbatim from [LPY23]

Definition 6 (Commitment). A commitment scheme is a pair of PPT ITMs $\langle C, R \rangle$ that satisfies the following. Let $m \in \{0, 1\}^{\ell(\lambda)}$ (where $\ell(\cdot)$ is some polynomial) is a message that C wants to commit to. The protocol consists of the following stages:

- **Commit Stage:** $C(m)$ and R interact with each other to generate a transcript τ , a decommitment decom as C 's private output, and a decision value $b_{\text{com}} \in \{\top, \perp\}$ as R 's private output. We denote this execution by $(\tau, \text{decom}, b_{\text{com}}) \leftarrow \langle C(m), R \rangle(1^\lambda)$. We say C is non-aborting if $b_{\text{com}} = \top$, and non-interactive if τ only consists a single message (from C to R).
- **Decommit Stage:** C sends the committed message m and the decommitment decom to R , and R outputs decision $b_{\text{dec}} \in \{\top, \perp\}$. We denote this execution by $b_{\text{dec}} \leftarrow \text{Verify}(\tau, m, \text{decom})$.¹² W.l.o.g., we assume that whenever R rejects at the end of the commit stage (i.e., $b_{\text{com}} = \perp$), R will reject at the end of decommit stage (i.e., $b_{\text{dec}} = \perp$). (Note that w.l.o.g., one can think that b_{com} is included in τ , because we can always modify the protocol to ask R to send b_{com} as the last-round message of the commit stage.)

The scheme satisfies the following completeness requirement:

1. **Completeness.** For any polynomial $\ell : \mathbb{N} \rightarrow \mathbb{N}$ and any $m \in \{0, 1\}^{\ell(\lambda)}$, it holds that

$$\Pr \left[b_{\text{com}} = b_{\text{dec}} = \top : \begin{array}{l} (\tau, \text{decom}, b_{\text{com}}) \leftarrow \langle C(m), R \rangle(1^\lambda) \\ b_{\text{dec}} \leftarrow \text{Verify}(\text{com}, m, \text{decom}) \end{array} \right] = 1.$$

Definition 7 (Computationally Hiding). A commitment scheme $\langle C, R \rangle$ is post-quantum computationally hiding if for any non-uniform QPT receiver $R^* = \{(R_\lambda^*, \rho_\lambda)\}_{\lambda \in \mathbb{N}}$ and any polynomial $\ell : \mathbb{N} \rightarrow \mathbb{N}$, $m_0, m_1 \in \{0, 1\}^{\ell(\lambda)}$, the following holds:

$$\{\text{OUT}_{R_\lambda^*} \langle C(m_0), R_\lambda^*(\rho_\lambda) \rangle(1^\lambda)\}_{\lambda, m_0, m_1} \stackrel{c}{\approx} \{\text{OUT}_{R_\lambda^*} \langle C(m_1), R_\lambda^*(\rho_\lambda) \rangle(1^\lambda)\}_{\lambda, m_0, m_1}$$

where $\text{OUT}_{R_\lambda^*} \langle C(m_b), R_\lambda^*(\rho_\lambda) \rangle(1^\lambda)$ ($b \in \{0, 1\}$) denotes the output of R_λ^* at the end of the commit stage, which can be quantum.

¹² We assume that there is no state information for R kept from the commit stage.

Definition 8 (Statistically Binding). A commitment scheme $\langle C, R \rangle$ is statistically binding if for any unbounded-time committer C^* , the following holds:

$$\Pr \left[\begin{array}{l} \exists m_0, m_1, \text{decom}_0, \text{decom}_1, \text{ s.t. } m_0 \neq m_1 \wedge \\ \text{Verify}(\tau, m_0, \text{decom}_0) = \text{Verify}(\tau, m_1, \text{decom}_1) = \top : (\tau, \text{decom}, b_{\text{com}}) \leftarrow \langle C^*, R \rangle(1^\lambda) \end{array} \right] = \text{negl}(\lambda).$$

Definition 9 (Committed Values). For a commitment scheme $\langle C, R \rangle$, we define the value function as follows:

$$\text{val}(\tau) := \begin{cases} m & \text{if } \exists \text{ unique } m \text{ s.t. } \exists \text{decom}, \text{Verify}(\tau, m, \text{decom}) = 1 \\ \perp & \text{otherwise} \end{cases},$$

where τ and decom are defined in the commit stage in [Def. 6](#).

Instantiation. It is well-known that post-quantum non-interactive statistically-binding commitment schemes satisfying [Def. 7](#) and [8](#) can be constructed from post-quantum secure injective one-way functions.

3.5.1 Non-Malleable Commitments

Post-Quantum Non-Malleable Commitments. This definition follows the formalization in [\[LPV08, GPR16\]](#). We consider a QPT man-in-the-middle adversary $\mathcal{M}$ interacting with a committer C in the *left*, and a receiver R in the *right*. We denote the relevant entities used in the right interaction as the “tilde’d” version of the corresponding entities on the left. In particular, suppose that C commits to m in the left interaction, and $\mathcal{M}$ commits to $\tilde{m}$ on the right, i.e., we set $\tilde{m} = \text{val}(\tilde{\tau})$ where $\tilde{\tau}$ is the transcript of the right session. Let $\text{mim}_{\langle C, R \rangle}^{\mathcal{M}}(\lambda, m, \rho)$ denote the random variable that is the pair $(\text{OUT}_{\mathcal{M}}, \tilde{m})$, consisting of $\mathcal{M}$ ’s output (could be a quantum state) as well as the value committed to by $\mathcal{M}$ on the right (assuming C commits to m on the left), where ρ is $\mathcal{M}$ ’s non-uniform quantum advice. We use a *tag-based* (or “identity-based”) specification, and ensure that $\mathcal{M}$ uses a distinct tag $\tilde{t}$ on the right from the tag t it uses on the left. This is done by stipulating that $\text{mim}_{\langle C, R \rangle}^{\mathcal{M}}(\lambda, m, \rho)$ outputs a special value $\perp_{\text{tag}}$ when $\mathcal{M}$ uses the same tag in both the left and right executions. The reasoning is that this corresponds to the uninteresting case when $\mathcal{M}$ is simply acting as a channel, forwarding messages from C on the left to R on the right and vice versa.

Definition 10 (Post-Quantum Non-Malleable Commitments). A commitment scheme $\langle C, R \rangle$ is said to be post-quantum non-malleable if for every (non-uniform) QPT man-in-the-middle adversary $\mathcal{M} = \{\mathcal{M}_\lambda, \rho_\lambda\}_{\lambda \in \mathbb{N}}$ and every polynomial $\ell : \mathbb{N} \rightarrow \mathbb{N}$, it holds that

$$\left\{ \text{mim}_{\langle C, R \rangle}^{\mathcal{M}_\lambda}(\lambda, m_0, \rho_\lambda) \right\}_{\lambda \in \mathbb{N}, m_0, m_1 \in \{0,1\}^{\ell(\lambda)}} \stackrel{c}{\approx} \left\{ \text{mim}_{\langle C, R \rangle}^{\mathcal{M}_\lambda}(\lambda, m_1, \rho_\lambda) \right\}_{\lambda \in \mathbb{N}, m_0, m_1 \in \{0,1\}^{\ell(\lambda)}}$$

where “ $\stackrel{c}{\approx}$ ” refers to computational indistinguishability against QPT distinguishers.

Remark 1 (On Entangled Auxiliary Information.). The above definition does not consider entanglement between $\mathcal{M}$ ’s auxiliary input and distinguisher’s auxiliary input. However, [\[BLS22, Claim 3.1\]](#) shows that the above definition implies the version that considers such entanglement.

3.5.2 k-Robustness

We also define *robust* non-malleable commitment scheme following [\[LP09\]](#). Roughly speaking, k -robust non-malleability means that non-malleability holds even if the adversary $\mathcal{M}$ interacts with an arbitrary external k -round interactive machine on the left. The formal definition is given below.

Definition 11 (Post-Quantum k -Robust Non-Malleable Commitments). A non-malleable commitment scheme $\langle C, R \rangle$ is said to be a post-quantum k -robust non-malleable if it satisfies the following in addition to post-quantum computational hiding, statistical binding and post-quantum non-malleability.

- **k -Robust Non-Malleability** For every (non-uniform) QPT adversary $\mathcal{A} = \{\mathcal{A}_\lambda, \rho_\lambda\}_{\lambda \in \mathbb{N}}$ and every polynomial $\ell : \mathbb{N} \rightarrow \mathbb{N}$, and k -round PPT interactive machine B , if

$$\{\langle B(m_0), \mathcal{A}_\lambda(\rho_\lambda) \rangle\}_{\lambda \in \mathbb{N}, m_0, m_1 \in \{0,1\}^{\ell(\lambda)}} \stackrel{c}{\approx} \{\langle B(m_1), \mathcal{A}_\lambda(\rho_\lambda) \rangle\}_{\lambda \in \mathbb{N}, m_0, m_1 \in \{0,1\}^{\ell(\lambda)}}$$

then it holds that, for every (non-uniform) QPT man-in-the-middle adversary $\mathcal{M} = \{\mathcal{M}_\lambda, \rho_\lambda\}_{\lambda \in \mathbb{N}}$

$$\{\text{mim}_{\langle C, R \rangle}^{B, \mathcal{M}_\lambda}(\lambda, m_0, \rho_\lambda)\}_{\lambda \in \mathbb{N}, m_0, m_1 \in \{0,1\}^{\ell(\lambda)}} \stackrel{c}{\approx} \{\text{mim}_{\langle C, R \rangle}^{B, \mathcal{M}_\lambda}(\lambda, m_1, \rho_\lambda)\}_{\lambda \in \mathbb{N}, m_0, m_1 \in \{0,1\}^{\ell(\lambda)}}$$

where $\text{mim}_{\langle C, R \rangle}^{B, \mathcal{M}_\lambda}(\lambda, m, \rho)$ denote the random variable that is the pair $(\text{OUT}_{\mathcal{M}_\lambda}, \tilde{m})$, consisting of $\mathcal{M}$'s output (could be a quantum state) as well as the value committed to by $\mathcal{M}$ on the right (using scheme $\langle C, R \rangle$), assuming B commits to m on the left, where ρ is $\mathcal{M}$'s non-uniform quantum advice.

3.5.3 First-Message Binding

We need an additional property for non-malleable commitments called *first-message binding*, which is first defined in [BLS22]¹³. It asserts that the first message of the sender determines the committed value. We recall its definition here.

Definition 12 (First-Message Binding). Let NMCom be a non-malleable commitment scheme. We say NMCom is *first-message binding* if it satisfies the following:

1. If the first message of NMCom is a receiver message, then it is *public-coin*—i.e., it consists solely of uniformly random strings.
2. With overwhelming probability over the choice of the honest receiver randomness, the first sender message binds the value of the commitment.

Instantiation. Post-quantum constant-round non-malleable commitment schemes satisfying Def. 11 and 12 are known from post-quantum OWFs [LPY23].

3.6 Interactive Proofs and Arguments

We define interactive proofs and arguments. An interactive protocol consists of PPT ITMs P referred to as a prover and V referred to as a verifier. We denote by $\langle P(x_P), V(x_V) \rangle(x)$ an execution of the protocol where x is a common input, x_P is P 's private input, and x_V is V 's private input. We denote by $\text{OUT}_V \langle P(x_P), V(x_V) \rangle(x)$ the final output of V in the execution. An honest verifier's output is $\top$ indicating acceptance or $\perp$ indicating rejection, and a quantum malicious verifier's output may be an arbitrary quantum state.

Definition 13 (PQ-IP and PQ-IA). A post-quantum interactive proof (PQ-IP) (resp. argument (PQ-IA)) for an NP language $\mathcal{L}$ is an interactive protocol $\langle P, V \rangle$ satisfying the following:

- **Completeness:** There exists a negligible function negl such that for any $\lambda \in \mathbb{N}$, $x \in L \cap \{0,1\}^\lambda$, and $w \in \mathcal{R}_\mathcal{L}(x)$, it holds that

$$\Pr[\text{OUT}_V \langle P(w), V \rangle(x) = \top] = 1 - \text{negl}(\lambda).$$

- **Statistical (resp. Computational) Soundness:** For any unbounded-time (resp. non-uniform QPT) machine $\{P_\lambda^*, \rho_\lambda\}_{\lambda \in \mathbb{N}}$, there exists a negligible function negl such that for any $\lambda \in \mathbb{N}$ and any $x \in \{0,1\}^\lambda \setminus \mathcal{L}$, it holds that

$$\Pr[\text{OUT}_V \langle P_\lambda^*(\rho_\lambda), V \rangle(x) = \top] \leq \text{negl}(\lambda).$$

Definition 14 (PQ- ε -ZKP and PQ- ε -ZKA). A post-quantum ε -simulatable zero-knowledge proof (PQ- ε -ZKP) (resp. argument (PQ- ε -ZKA)) is a PQ-IP (resp. PQ-IA) that additionally satisfies the following:

¹³ The original definition of first-message binding in [BLS22] focuses on extractable commitment, but it naturally extends to non-malleable commitments

- **ε -Simulatable Zero Knowledge:** For any non-uniform QPT machine $\{V_\lambda^*, \rho_\lambda\}_{\lambda \in \mathbb{N}}$, we have

$$\{\text{OUT}_{V_\lambda^*} \langle P(w), V_\lambda^*(\rho_\lambda) \rangle(x)\}_{\lambda, x, w} \stackrel{c}{\approx}_\varepsilon \{\text{OUT}_{V_\lambda^*} \left(\mathcal{S}^{V_\lambda^*(x, \rho_\lambda)}(x) \right)\}_{\lambda, x, w}$$

where $\lambda \in \mathbb{N}$, $x \in \mathcal{L} \cap \{0, 1\}^\lambda$, and $w \in \mathcal{R}_\mathcal{L}(x)$.

Post-Quantum ε -Simulatable Zero Knowledge Argument of Knowledge (PQ- ε -ZKAoK) Next, let us review the definition of PQ- ε -ZKAoK. The following definition is a combination of [LMS22, Definition 1.5] and [LPY23, Definition 14].

Definition 15 (PQ- ε -ZKAoK *without* post-extraction simulation). A post-quantum ε -simulatable zero knowledge argument of knowledge (PQ-ZKAoK) *without* post-extraction simulation is a PQ-ZKA that additionally satisfies the following:

- **Knowledge Soundness** There is an oracle-aided QPT algorithm $\mathcal{K}$, a polynomial poly , a negligible function negl , and a constant $d \in \mathbb{N}$ such that for any QPT malicious prover $P^* = \{P_\lambda^*, \rho_\lambda\}_{\lambda \in \mathbb{N}}$, $\lambda \in \mathbb{N}$ and $x \in \{0, 1\}^\lambda$, we have

$$\Pr[w' \in \mathcal{R}_\mathcal{L}(x) : w' \leftarrow \mathcal{K}^{P_\lambda^*(\rho_\lambda)}(x)] \geq \frac{1}{\text{poly}(\lambda)} \cdot \Pr[\text{OUT}_V \langle P_\lambda^*(\rho_\lambda), V \rangle(x) = \top]^d - \text{negl}(\lambda).$$

Instantiation. Post-quantum constant-round ε -simulatable zero knowledge argument of knowledge (PQ-ZKAoK) without post-extraction simulation, satisfying Def. 15, can be constructed from post-quantum OWFs [CCLY22].

Definition 16 (PQ-WIP and PQ-WIA). A post-quantum witness indistinguishable proof (PQ-WIP) (resp. argument (PQ-WIA)) is a PQ-IP (resp. PQ-IA) that additionally satisfies the following:

- **Witness Indistinguishability:** For any non-uniform QPT machine $\{V_\lambda^*, \rho_\lambda\}_{\lambda \in \mathbb{N}}$, we have

$$\{\text{OUT}_{V_\lambda^*} \langle P(w_0), V_\lambda^*(\rho_\lambda) \rangle(x)\}_{\lambda, x, w_0, w_1} \stackrel{c}{\approx} \{\text{OUT}_{V_\lambda^*} \langle P(w_1), V_\lambda^*(\rho_\lambda) \rangle(x)\}_{\lambda, x, w_0, w_1}$$

where $\lambda \in \mathbb{N}$, $x \in \mathcal{L} \cap \{0, 1\}^\lambda$, and $w_0, w_1 \in \mathcal{R}_\mathcal{L}(x)$. Statistical witness indistinguishability is defined analogously when considering statistical indistinguishability instead of plain computational indistinguishability.

Instantiation. Post-Quantum constant-round witness indistinguishable arguments satisfying Def. 16 are known from the polynomial hardness of PQ-OWF (which is just a standard sigma-protocol.).

Definition 17 (PQ-sWIP and PQ-sWIA). A post-quantum strong witness indistinguishable proof (PQ-sWIP) (resp. argument (PQ-sWIA)) is a PQ-IP (resp. PQ-IA) that additionally satisfies the following:

- **Strong Witness Indistinguishability:** For every $\{\mathcal{X}_\lambda^0, \mathcal{W}_\lambda^0\}_{\lambda \in \mathbb{N}}$, $\{\mathcal{X}_\lambda^1, \mathcal{W}_\lambda^1\}_{\lambda \in \mathbb{N}}$ and $\{z_\lambda\}_{\lambda \in \mathbb{N}}$ where each $\mathcal{X}_\lambda^b, \mathcal{W}_\lambda^b$ is a joint distribution that ranges over $\mathcal{R}_\mathcal{L} \cap (\{0, 1\}^\lambda \times \{0, 1\}^*)$ and each z_λ is a string in $\{0, 1\}^*$, if it holds

$$\{\mathcal{X}_\lambda^0\}_{\lambda \in \mathbb{N}} \stackrel{c}{\approx} \{\mathcal{X}_\lambda^1\}_{\lambda \in \mathbb{N}}$$

For any non-uniform QPT machine $\{V_\lambda^*, \rho_\lambda\}_{\lambda \in \mathbb{N}}$ and $\lambda \in \mathbb{N}$, it holds that

$$\{\text{OUT}_{V^*} \langle P(w_0), V^*(z_\lambda) \rangle(x_0)\}_{\lambda, (x_0, w_0) \leftarrow (\mathcal{X}_\lambda^0, \mathcal{W}_\lambda^0)} \stackrel{c}{\approx} \{\text{OUT}_{V^*} \langle P(w_1), V^*(z_\lambda) \rangle(x_1)\}_{\lambda, (x_1, w_1) \leftarrow (\mathcal{X}_\lambda^1, \mathcal{W}_\lambda^1)}$$

4 Post-Quantum Constant-Round Concurrent 2PC

We construct a post-quantum constant-round concurrent 2PC, $\Pi(\mathcal{F})$, from standard assumptions. Its description is given in [Prot. 2](#). This construction is based on the following building blocks:

- A non-interactive statistically-binding commitment scheme Com ; We use $\text{Com}(m; r)$ to denote the message where a string m is committed using randomness r ¹⁴.
- A constant-round ε -simulatable zero-knowledge argument of knowledge ε -ZKAoK without post-extraction simulation (as per [Def. 15](#)).
- A constant-round witness indistinguishable argument WI. (as per [Def. 16](#)).
- A constant-round non-malleable commitment scheme NMCOM with first-message binding property and k -robustness. (as per [Def. 11](#) and [12](#)).
- A constant-round semi-honest two party computation protocol $\langle P_A^{\text{sh}}, P_B^{\text{sh}} \rangle$ that securely computes $\mathcal{F}$. (as per [Def. 2](#)).

Protocol 2: Post-Quantum Constant-Round Concurrent 2PC

Parameter: Let k be $\text{MAX}(\text{rc}(\varepsilon\text{-ZKAoK})+1, 2*t*(1+\text{rc}(\text{WI}))+2)$, where $\text{rc}(\cdot)$ denotes the round complexity.

Phase I: Trapdoor Generation

1. P_A uniformly samples a random string τ_A where $\tau_A \xleftarrow{\$} \{0, 1\}^\lambda$. P_A samples a random tape r_A^τ and sends $c_A^\tau = \text{Com}(\tau_A; r_A^\tau)$ to P_B .
2. P_B acts symmetrically as P_A did in [Step 1](#) of **Phase I**. That is, P_B sends $c_B^\tau = \text{Com}(\tau_B; r_B^\tau)$ to P_A .
3. P_A samples a random tape r_A^{trap} and sends $c_A^{\text{trap}} = \text{Com}(0; r_A^{\text{trap}})$ to P_B .
4. (a) P_A samples a random tape r_A^{prove} and sends $c_A^{\text{prove}} = \text{Com}(0; r_A^{\text{prove}})$ to P_B .
 (b) P_A proves to P_B that it “knows” the value committed in c_A^{prove} using ε -ZKAoK. Formally, P_A proves that $c_A^{\text{prove}} \in \mathcal{L}_{\text{prove}}$, where

$$\mathcal{L}_{\text{prove}} := \{c_A^{\text{prove}} \mid \exists(m, r) \text{ s.t. } c_A^{\text{prove}} = \text{Com}(m; r)\}. \quad (1)$$

Note that P_A uses the $(0, r_A^{\text{prove}})$ as the witness when executing this ε -ZKAoK.

- (c) P_B sends τ_B and r_B^τ to P_A as the decommit information for c_B^τ . P_A aborts if the decommitment is not valid.
- (d) (**NMCOM.**) P_A and P_B engage in an execution of NMCOM, where P_A commits to the value r_A^{trap} defined in [Step 3](#). Note that the value r_A^{trap} serves as a valid witness for the statement that “the c_A^{trap} in [Step 3](#) of **Phase I** is a commitment to the value 0.” Formally, r_A^{trap} is a valid witness for the fact that $c_A^{\text{trap}} \in \mathcal{L}_{\text{Com}(0)}$ where

$$\mathcal{L}_{\text{Com}(0)} := \{c \mid \exists r \text{ s.t. } c = \text{Com}(0; r)\}. \quad (2)$$

We use c_A^{NM} to refer to (the transcript of) this NMCOM.

- (e) (**WI.**) P_A and P_B engage in an execution of WI, where P_A proves that *either*
 - i. (**honest statement.**) P_A behaves honestly in [Step 4d](#) (namely, c_A^{NM} is indeed a valid NMCOM to the value r_A^{trap} , which is a valid witness for the fact that $c_A^{\text{trap}} \in \mathcal{L}_{\text{Com}(0)}$), *or*
 - ii. (**trapdoor statement.**) the c_A^{prove} sent by P_A in [Step 4a](#) is a commitment to the value τ_B defined in [Step 4c](#).

Note that when executing this WI, the honest P_A uses the witness corresponding to the **honest statement** above.

¹⁴ Similar to [\[GGJS12\]](#), we use a non-interactive, statistically binding commitment scheme in the presentation of our protocol for simplicity. Alternatively, one can replace it with Naor’s interactive scheme by requiring both parties to send the first message of Naor’s commitment at the onset of the protocol.

5. P_B acts symmetrically as P_A did in [Step 3](#) of this phase. (That is, P_B samples a random tape r_B^{trap} and sends $\mathbf{c}_B^{\text{trap}} = \text{Com}(0; r_B^{\text{trap}})$ to P_A .)
6. Similarly, P_B acts symmetrically as P_A did in [Step 4](#) of this phase.

Phase II: Input Commitment

1. P_A samples r'_A (which will be used in the secure computation phase). It sends $\mathbf{c}_A = \text{Com}((x_A, r'_A); r_A^{\text{com}})$
2. Similarly, P_B acts symmetrically. P_B samples r'_B and sends $\mathbf{c}_B = \text{Com}((x_B, r'_B); r_B^{\text{com}})$

Phase III: Secure Computation

1. P_A samples r''_B and sends it to P_B , where $|r''_B| = |r'_B|$. Similarly, P_B samples r''_A and sends it to P_A . Let $r_A = r'_A \oplus r''_A$ and $r_B = r'_B \oplus r''_B$. r_A and r_B are the random coins that P_A and P_B will use during the execution of $\langle P_A^{\text{sh}}, P_B^{\text{sh}} \rangle$.
2. Let t be the number of rounds in $\langle P_A^{\text{sh}}, P_B^{\text{sh}} \rangle$, where one round consists of a message from P_A^{sh} followed by a reply from P_B^{sh} . Let transcript $T_{A,j}$ (resp., $T_{B,j}$) be defined to contain all the messages exchanged between P_A^{sh} and P_B^{sh} before the point P_A^{sh} (resp., P_B^{sh}) is supposed to send a message in round j . For $j = 1, \dots, t$:
 - (a) $P_A \Rightarrow P_B$: Compute $m_{A,j} = P_A^{\text{sh}}(T_{A,j}, x_A, r_A)$ and send it to P_B . P_A and P_B then engage in an execution of WI where P_A proves that *either*
 - **(honest statement)** $\mathbf{c}_A$ is a commitment of $\widehat{x}_A, \widehat{r}_A'$ such that $m_{A,j} = P_A^{\text{sh}}(T_{A,j}, \widehat{x}_A, \widehat{r}_A' \oplus r''_A)$, *or*
 - **(trapdoor statement)** $\mathbf{c}_A^{\text{trap}}$ is a commitment to bit 1.

Formally, P_A proves that $((m_{A,j}, \mathbf{c}_A), \mathbf{c}_A^{\text{trap}}) \in \mathcal{L}_{\text{sh}}^{T_{A,j}} \vee \mathcal{L}_{\text{trap}}$, and $\mathcal{L}_{\text{sh}}^{T_{A,j}} \vee \mathcal{L}_{\text{trap}}$ denotes the OR-composed language, where

$$\mathcal{L}_{\text{sh}}^{T_{A,j}} := \left\{ (m_{A,j}, \mathbf{c}_A) \mid \bigwedge \begin{array}{l} \exists \widehat{x}_A, \widehat{r}_A', r \text{ s.t. } \mathbf{c}_A = \text{Com}((\widehat{x}_A, \widehat{r}_A'); r) \\ m_{A,j} = P_A^{\text{sh}}(T_{A,j}, \widehat{x}_A, \widehat{r}_A' \oplus r''_A) \end{array} \right\}. \quad (3)$$

and

$$\mathcal{L}_{\text{trap}} := \{ \mathbf{c}_A^{\text{trap}} \mid \exists r \text{ s.t. } \mathbf{c}_A^{\text{trap}} = \text{Com}(1; r) \}. \quad (4)$$

Note that P_A uses $((x_A, r'_A, r_A^{\text{com}}), \perp)$ as the witness when executing this WI.

- (b) $P_B \Rightarrow P_A$: P_B now acts symmetrically.

4.1 Proof of Security

Theorem 3. $\Pi(\mathcal{F})$ ([Prot. 2](#)) UC-SPS-realizes $\mathcal{F}$ for every well-formed functionality $\mathcal{F}$.

Proof. Note that we aim to prove security in the concurrent setting, where an (unbounded) polynomial number of sessions (also referred to as *instances*) of [Prot. 2](#) may execute concurrently. The adversary is allowed to corrupt exactly one party per session (without loss of generality). We emphasize that this is an “interchangeable-role” corruption model, meaning the adversary may corrupt, for example, party P_A in one session and party P_B in another.

We now describe the simulator. The simulator, dubbed $\mathcal{S}$, operates similarly across all sessions in the concurrent execution. Therefore, it suffices to specify its behavior in a single session (keeping in mind that the simulator is simultaneously handling all sessions in the concurrent setting). Moreover, we assume that in this specific session, P_B is corrupted. The simulator’s behavior is symmetric in sessions where P_A is corrupted instead. We present the simulator in [Algo. 4.1](#).

Algorithm 4.1: Simulator $\mathcal{S}$ (in a session where P_B is corrupted):

Phase I: Trapdoor Generation

1. In [Step 1](#), $\mathcal{S}$ follows the protocol specification and behaves exactly like the honest P_A . Namely, it generates and sends c_A^τ honestly to P_B , and receives c_B^τ . However, after receiving c_B^τ , $\mathcal{S}$ extracts τ_B by running in super-polynomial time.
2. If the extraction is successful, in [Step 3](#), $\mathcal{S}$ commits to 1 in $c_A^{\text{trap}} = \text{Com}(1; r_A^{\text{trap}})$ instead of 0. Otherwise, $\mathcal{S}$ outputs $\perp$.
3. In [Step 4a](#), $\mathcal{S}$ commits to τ_B (instead of 0) in c_A^{prove} .
4. In [Step 4b](#), $\mathcal{S}$ follows the protocol with an honest execution of ε -ZKAoK to prove knowledge of the decommitment value of c_A^{prove} (use $(\tau_B, r_A^{\text{prove}})$ as witness).
5. In [Step 4c](#), $\mathcal{S}$ acts honestly and check the decommit value of c_B^τ sent by malicious P_B . If the decommit value is incorrect, $\mathcal{S}$ aborts.
6. In [Step 4d](#), $\mathcal{S}$ commits to 0 (instead of the value r_A^{trap}).
7. In [Step 4e](#), $\mathcal{S}$ participates in the execution of WI, using the witness for the **trapdoor statement** defined in [Step 4e](#) in [Prot. 2](#). Note that $\mathcal{S}$ commits to τ_B in [Step 3](#) above, therefore the witness for the **trapdoor statement** is available to $\mathcal{S}$.

Phase II: Input Commitment

$\mathcal{S}$ commits to string of all zeros instead of $x_A || r'_A$ in c_A , and after receiving c_B , $\mathcal{S}$ extracts $x_B || r'_B$ by running in super-polynomial time. If the extraction fails, $\mathcal{S}$ outputs $\perp$.

Phase III: Secure Computation Phase

Let $\mathcal{S}_{\text{sh}}$ denote the simulator for the semi-honest protocol $\langle P_A^{\text{sh}}, P_B^{\text{sh}} \rangle$ that securely computes $\mathcal{F}$. $\mathcal{S}$ internally emulates $\mathcal{S}_{\text{sh}}$ against a malicious P_B , using the x_B (extracted in the last step via brute force) as P_B 's input. When $\mathcal{S}_{\text{sh}}$ makes a query to the ideal functionality $\mathcal{F}$, $\mathcal{S}$ forwards the query (along with its session ID) to the (external) ideal functionality $\mathcal{F}$, and returns the output to $\mathcal{S}_{\text{sh}}$. $\mathcal{S}_{\text{sh}}$ outputs a transcript

$$(m_{A,1}, m_{B,1}, \dots, m_{A,t}, m_{B,t})$$

and an associated randomness r_B for malicious party P_B . Then, the simulator $\mathcal{S}$ now performs the following steps:

1. In [Step 1](#) of **Phase III**, $\mathcal{S}$ sends $r_B'' := r_B \oplus r'_B$ to P_B (instead of sampling it randomly).
2. For each round $i \in [t]$:
 - (a) $\mathcal{S}$ sends $m_{A,i}$ to P_B
 - (b) $\mathcal{S}$ uses the trapdoor witness r_A^{trap} when it participates in the WI in [Step 2](#) of **Phase III**. Note that $\mathcal{S}$ commits to 1 in c_A^{trap} using randomness r_A^{trap} in the simulated view.
 - (c) On receiving P_B 's message $\widetilde{m_{B,i}}$, $\mathcal{S}$ checks if $\widetilde{m_{B,i}} = m_{B,i}$. $\mathcal{S}$ outputs $\perp$ if $\widetilde{m_{B,i}} \neq m_{B,i}$, otherwise it continues.
 - (d) $\mathcal{S}$ act as an honest verifier when it engages with P_B in an execution of WI, and output $\perp$ if the proof is not valid.

In order to argue the indistinguishability between the real world execution and the simulated interaction produced by $\mathcal{S}$ running in super-polynomial time, we first consider the following two hybrids.

- **Hybrid H_0** : This is the real experiment where m sessions (aka instances) of [Prot. 2](#) are running concurrently. The adversary $\mathcal{A}_{\Pi(\mathcal{F})}$ corrupts exactly one party in each session. The output of this hybrid consists the output of the environment Z , the output of the honest parties and view of the adversary $\mathcal{A}_{\Pi(\mathcal{F})}$.
- **Hybrid H_1** : This corresponds to the simulated execution of the ideal world where $\mathcal{S}$ simulates the honest parties (using the strategy defined in [Algo. 4.1](#)). The honest parties, in each session, receive input from the environment, query the ideal functionality on their input and output the response they receive from the ideal functionality as their output. Again, the output of this hybrid consists the output of the environment Z , the output of the honest parties and view of the adversary $\mathcal{A}_{\Pi(\mathcal{F})}$.

Next, we define a sequence of hybrids $H_{i:j}$ where $i \in [2m]$ and $j \in [6]$. We also define two additional hybrids $H_{0:6}$ and $H_{2m+1:0}$, where $H_{0:6}$ is equivalent to H_0 and $H_{2m+1:0}$ is equivalent to H_1 . Before we give the descriptions of these hybrids, we give some additional notation that will be useful later.

Special Messages. As mentioned above, we use m to denote the total number of concurrent sessions. Without loss of generality, we use a single number $i \in [m]$ to denote the session identification number (sid).

For every session with sid $i \in [m]$, we define two special messages:

- **The 1st special message:** This is the commitment to the random string τ made by the corrupted party in session i . That is:
 - If P_A is corrupted in session i , then the 1st special special message is the c_A^τ sent by the malicious P_A in [Step 1](#) of **Phase I** in of [Prot. 2](#).
 - if P_B is corrupted in session i , then the 1st special special message is the c_B^τ sent by the malicious P_B in [Step 2](#) of **Phase I** in of [Prot. 2](#).
- **The 2nd special message:** This is the **Phase-II** commitment to the input and randomness made by the corrupted party in session i . That is:
 - If P_A is corrupted in session i , then the 2nd special special message is the c_A sent by the malicious P_A in [Step 1](#) of **Phase II** in of [Prot. 2](#).
 - If P_B is corrupted in session i , then the 2nd special special message is the c_B sent by the malicious P_B in [Step 2](#) of **Phase II** in of [Prot. 2](#).

In the following, for simplicity of notation, we will refer the honest party in every session as H and the malicious party in every session as $\mathcal{A}$. We consider two special commitments c_A^τ and c_A sent by $\mathcal{A}$, and refer them as $c_A^{\tau,i}$ and c_A^i (with sid in superscript). Note that there are exactly two special commitments in every session. We consider a numbered ordering of all the $2m$ commitments across the m sessions. In the following, we use $s(i)$ to denote the sid of the protocol session that contains i^{th} special commitment; Note that $c_A^{\tau,i}$ always appears before c_A^i for any sid $i \in [m]$, but not necessarily consecutively.

Invariant Condition. Similar to [\[GGJS12\]](#), we need to prove that the trapdoor condition of WI in the secure computation phase is false for every session and all intermediate hybrids. More formally, we need to show that, for all sid $i \in [m]$, $c_A^{\text{trap}} \notin \mathcal{L}_{\text{trap}}$ in [Language \(4\)](#). Therefore, c_A^{NM} must be a valid non-malleable commitment to the value r_A^{trap} . We define m variable $\{v_{i:j}^\ell\}_{\ell=1}^m$ for each intermediate hybrid, where $v_{i:j}^\ell$ is the value committed in c_A^{NM} in session sid ℓ , hybrid $H_{i:j}$, and we need to show that $v_{i:j}^\ell$ is a valid decommitment of $c_A^{\text{trap},\ell}$ to the value $r_A^{\text{trap},\ell}$, for every session $\ell \in [m]$.

Now, before we give the description of our hybrids, and argue that the invariant condition still holds as we move from one hybrid to another, we first claim that the invariant condition holds in the real execution.

Lemma 1. *In hybrid H_0 which corresponds to the real execution of [Prot. 2](#), for every $\ell \in [m]$, it holds that $v_0^\ell = r_A^{\text{trap},\ell}$. That is, c_A^{NM} is indeed a valid NMCOM to the value r_A^{trap} , which is a valid witness for the fact that $c_A^{\text{trap}} \in \mathcal{L}_{\text{Com}(0)}$.*

Proof. We first show that the trapdoor statement defined in [Step 4e](#) is false, for every $\ell \in [m]$. For the sake of contradiction, assume there exists an $\ell \in [m]$ such that the trapdoor statement for $\mathcal{A}$ is true, namely, $c_A^{\text{prove},\ell}$ is a commitment of τ_H^ℓ in session ℓ . Levering this $\mathcal{A}$, we construct in [Algo. 4.2](#) a non-uniform QPT algorithm $\mathcal{A}^*$ breaks the hiding of Com.

Algorithm 4.2: $\mathcal{A}^*$ against hiding of Com
$\mathcal{A}'$ internally emulates the algorithm $\mathcal{A}$, while externally interacting with a challenger of the hiding property of Com. In particular, $\mathcal{A}^*$ proceeds as follows. 1. At the beginning, $\mathcal{A}^*$ makes a random guess $\ell^* \in [m]$ about the session number where the trapdoor statement for $\mathcal{A}$ is true. 2. $\mathcal{A}^*$ follows the Prot. 2 using honest parties' inputs (received as a non-uniform advice) in every session $\ell \in [m]$ where $\ell \neq \ell^*$

3. $\mathcal{A}^*$ samples $\tau_{\mathcal{A}^*,0}^{\ell^*}, \tau_{\mathcal{A}^*,1}^{\ell^*} \xleftarrow{\$} \{0,1\}^\lambda$ and sends $\tau_{\mathcal{A}^*,0}^{\ell^*}$ and $\tau_{\mathcal{A}^*,1}^{\ell^*}$ to the external challenger.
4. $\mathcal{A}^*$ receives $c_{\mathcal{A}^*}^{\tau, \ell^*}$ from the external challenger.
5. In session ℓ^* , instead of generates c_H^{τ, ℓ^*} honestly, $\mathcal{A}^*$ sends $c_{\mathcal{A}^*}^{\tau, \ell^*}$ it received from external challenger to $\mathcal{A}$, and follows the [Prot. 2](#) until $\mathcal{A}$ sends $c_{\mathcal{A}}^{\text{prove}, \ell^*}$ to $\mathcal{A}^*$ (as the honest party) in [Step 4a](#).
6. Once $\mathcal{A}$ starts its ε -ZKAoK with $\mathcal{A}^*$ (as the honest party) in [Step 4b](#), $\mathcal{A}^*$ invokes $\mathcal{K}$, the knowledge extractor for ε -ZKAoK (defined in [Def. 15](#)), to extract the witness that $\mathcal{A}$ used when executing ε -ZKAoK in [Step 4b](#), which is also the value committed in $c_{\mathcal{A}}^{\text{prove}, \ell^*}$. If the extraction succeeds, then $\mathcal{A}^*$ outputs 0 if $\text{val}(c_{\mathcal{A}}^{\text{prove}, \ell^*}) = \tau_{\mathcal{A}^*,0}^{\ell^*}$, and outputs 1 if $\text{val}(c_{\mathcal{A}}^{\text{prove}, \ell^*}) = \tau_{\mathcal{A}^*,1}^{\ell^*}$; Otherwise outputs a random bit.

Clearly, $\mathcal{A}$ runs in polynomial time, and notice that $\tau_{\mathcal{A}^*,0}^{\ell^*}$ and $\tau_{\mathcal{A}^*,1}^{\ell^*}$ are sampled uniformly, therefore, from $\mathcal{A}$'s perspective, this is exactly the same execution as [Prot. 2](#), and thus by assumption, there exists an $\ell \in [m]$ such that the trapdoor statement for $\mathcal{A}$ is true with non-negligible probability. The probability that $\ell^* = \ell$, is $1/m$, then by the knowledge soundness of ε -ZKAoK, $\mathcal{K}$ returns $\tau_{\mathcal{A}^*,0}^{\ell^*}$ or $\tau_{\mathcal{A}^*,1}^{\ell^*}$ is non-negligible. This breaks the hiding of Com and therefore leads to a contradiction.

Since we just showed that the trapdoor statement is false, for every $\ell \in [m]$, by the soundness of WI , the honest statement must be true with overwhelming probability, namely $c_{\mathcal{A}}^{\text{NM}}$ is indeed a valid NMCOM to the value $r_{\mathcal{A}}^{\text{trap}}$, which is a valid witness for the fact that $c_{\mathcal{A}}^{\text{trap}} \in \mathcal{L}_{\text{Com}(0)}$. Therefore, for every $\ell \in [m]$, it holds that $v_0^\ell = r_{\mathcal{A}}^{\text{trap}, \ell}$. $\square$

Description of $H_{i,j}$: Now, we are ready to describe our sequence of hybrids $H_{i,j}$, where $j \in [6]$ and $i \in [2m]$, which corresponds the numbered ordering of all $2m$ special commitments across m sessions. Recall that we use $s(i)$ to denote the session number of the i_{th} special commitment.

- Hybrid $H_{i,1}$: This is identical to $H_{i-1,6}$ (for every $i \in [2m]$) except that it runs in super-polynomial time to extract values from the i_{th} special commitment across all m sessions. Note that it runs in polynomial time after extracting the i_{th} special commitment. If extraction fails, it outputs $\perp$, and otherwise works similarly to the previous hybrid. We note that this experiment obtains values from the i_{th} special commitment but does not use it. Thus, by the statistical binding property of Com , we have

$$H_{i,1} \stackrel{s}{\approx} H_{i-1,6}. \quad (5)$$

$$\forall \ell \in [m], v_{i,1}^\ell \stackrel{s}{\approx} v_{i-1,6}^\ell \quad (6)$$

Remark 2. An alternative way to interpret hybrid $H_{i,*}$ is to consider a “super-polynomial time line” placed right after the i_{th} special commitment, and $H_{i,*}$ runs in super-polynomial time to extract values from the first i (out of $2m$) special commitments above this line, and runs in polynomial time afterwards.

- Hybrid $H_{i,2}$: This is identical to $H_{i,1}$ except if the i_{th} special commitment corresponds to $c_{\mathcal{A}}^{\tau, s(i)}$ for session $s(i)$, then $\mathcal{S}$ follows [Steps 3 to 7](#) in the trapdoor generation phase of our description of the final simulator, using $\tau_{\mathcal{A}}^{s(i)}$ extracted from $c_{\mathcal{A}}^{\tau, s(i)}$ for session $s(i)$.
In more detail, in the trapdoor generation phase of session $s(i)$:

1. In [Step 1](#), $\mathcal{S}$ commits to $\tau_{\mathcal{A}}^{s(i)}$ instead of 0 in $c_H^{\text{prove}, s(i)}$;
2. In [Step 4b](#), $\mathcal{S}$ follows the protocol with an honest execution of ε -ZKAoK to prove knowledge of the decommitment value of $c_H^{\text{prove}, s(i)}$ (use $(\tau_{\mathcal{A}}^{s(i)}, r_H^{\text{prove}, s(i)})$ as witness).
3. In [Step 4c](#), $\mathcal{S}$ acts honestly and check the decommit value of $c_{\mathcal{A}}^{\tau, s(i)}$ sent by malicious $P_{\mathcal{A}}$. If the decommit value is incorrect, $\mathcal{S}$ aborts.
4. In [Step 4d](#), $\mathcal{S}$ commits to 0 instead of decommitment of $c_H^{\text{trap}, s(i)}$ in $c_H^{\text{MN}, s(i)}$.
5. In [Step 4e](#), $\mathcal{S}$ participates in the execution of WI using $(\perp, r_H^{\text{prove}, s(i)})$ as the witness. Note that $\mathcal{S}$ commits to $\tau_{\mathcal{A}}^{s(i)}$ in the simulated view, therefore the (trapdoor) witness corresponding to the second part of the statement is available to $\mathcal{S}$.

Claim 4.

$$H_{i:2} \stackrel{c}{\approx} H_{i:1} \quad (7)$$

$$\forall \ell \in [m], v_{i:2}^\ell \stackrel{c}{\approx} v_{i:1}^\ell \quad (8)$$

Proof. We consider the following sequence of hybrids.

- Hybrid $H_{i:1:1}$: This is identical to $H_{i:1}$ except in session $s(i)$, $\mathcal{S}$ commits to $\tau_{\mathcal{A}}^{s(i)}$ instead of 0 in $\mathbf{c}_H^{\text{prove}, s(i)}$, and in [Step 4b](#) of session $s(i)$, $\mathcal{S}$ follows the protocol with an honest execution of ε -ZKAoK to prove knowledge of the decommitment value of $\mathbf{c}_H^{\text{prove}, s(i)}$ (use $(\tau_{\mathcal{A}}^{s(i)}, r_H^{\text{prove}, s(i)})$ as witness). Note that $\mathcal{S}$ finishes the super-polynomial time extraction before the start of [Step 1](#) in session $s(i)$ and because ε -Zero-Knowledge property imply strong witness-indistinguishability. Thus, by the non-uniform hiding property of Com and the strong non-uniform witness-indistinguishability property of ε -ZKAoK, we have

$$H_{i:1:1} \stackrel{c}{\approx} H_{i:1} \quad (9)$$

Additionally, in order to show $\forall \ell \in [m], v_{i:1:1}^\ell \stackrel{c}{\approx} v_{i:1}^\ell$, $\forall \ell \in [m]$, we consider the following two cases:

1. if the first message of $\mathbf{c}_{\mathcal{A}}^{\text{NM}, \ell}$ of session ℓ is sent before $\mathbf{c}_H^{\text{prove}, s(i)}$ of session $s(i)$, then by the first-message-binding property (defined in [Def. 12](#)) of NMCOM, we have $v_{i:1:1}^\ell = v_{i:1}^\ell$.
2. if the first message of $\mathbf{c}_{\mathcal{A}}^{\text{NM}, \ell}$ of session ℓ is sent after (or simultaneously) $\mathbf{c}_H^{\text{prove}, s(i)}$ of session $s(i)$. Recall that in session $s(i)$, $H_{i:1:1}$ only made modifications in $\mathbf{c}_H^{\text{prove}, s(i)}$ and ε -ZKAoK, then by the k -robust property of NMCOM (recall that we set $k \geq \text{rc}(\varepsilon\text{-ZKAoK}) + 1$), we have $v_{i:1:1}^\ell \stackrel{c}{\approx} v_{i:1}^\ell$.

Therefore, we have

$$\forall \ell \in [m], v_{i:1:1}^\ell \stackrel{c}{\approx} v_{i:1}^\ell \quad (10)$$

- Hybrid $H_{i:1:2}$: This is identical to $H_{i:1:1}$ except in [Step 4e](#), $\mathcal{S}$ participates in the execution of WI using trapdoor witness $(\perp, r_H^{\text{prove}, s(i)})$. Since ε -Zero-Knowledge property imply witness-indistinguishability, by the non-uniform witness-indistinguishability property of ε -ZKAoK, we have

$$H_{i:1:2} \stackrel{c}{\approx} H_{i:1:1} \quad (11)$$

Additionally, by the same proof for [Eq. \(10\)](#), we have

$$\forall \ell \in [m], v_{i:1:2}^\ell \stackrel{c}{\approx} v_{i:1:1}^\ell \quad (12)$$

- Hybrid $H_{i:1:3}$: This is identical to $H_{i:1:2}$ except in [Step 4d](#) session $s(i)$, $\mathcal{S}$ commits to 0 instead of decommitment of $\mathbf{c}_H^{\text{trap}, s(i)}$ in $\mathbf{c}_H^{\text{MN}, s(i)}$. Note that $\mathcal{S}$ finishes the super-polynomial time extraction before the start of [Step 4d](#) in session $s(i)$. Thus, by the non-uniform hiding property of NMCOM, we have

$$H_{i:1:3} \stackrel{c}{\approx} H_{i:1:2} \quad (13)$$

Additionally, by the same proof for [Eq. \(10\)](#), except we rely on standard non-malleability of NMCOM instead of k -robustness, we have

$$\forall \ell \in [m], v_{i:1:3}^\ell \stackrel{c}{\approx} v_{i:1:2}^\ell \quad (14)$$

□

- Hybrid $H_{i:3}$: This is identical to $H_{i:2}$ except if the i_{th} special commitment corresponds to $\mathbf{c}_{\mathcal{A}}^{\tau, s(i)}$ for session $s(i)$, then $\mathcal{S}$ commits to 1 instead of 0 in $\mathbf{c}_H^{\text{trap}, s(i)}$ in the trapdoor generation phase of session $s(i)$. Note

that $\mathcal{S}$ finishes the super-polynomial time extraction before the start of [Step 3](#) in session $s(i)$. Thus, by the non-uniform hiding property of Com , we have

$$H_{i:3} \stackrel{c}{\approx} H_{i:2} \quad (15)$$

Additionally, by a similar proof as for [Eq. \(10\)](#), we have

$$\forall \ell \in [m], v_{i:3}^\ell \stackrel{c}{\approx} v_{i:2}^\ell \quad (16)$$

- Hybrid $H_{i:4}$: This is identical to $H_{i:3}$ except if the i_{th} special commitment corresponds to $\mathbf{c}_{\mathcal{A}}^{\tau, s(i)}$ for session $s(i)$, then $\mathcal{S}$ use the trapdoor witness to finish **every** execution of WI in the secure computation phase. More specifically, use $\mathbf{c}_{\mathcal{A}}^{\text{trap}, s(i)} \in \mathcal{L}_{\text{trap}}$ in [Language \(4\)](#) as the witness to finish all WI instances in the secure computation phase. Since in $H_{i:3}$, $\mathcal{S}$ commits to 1 in $\mathbf{c}_H^{\text{trap}, s(i)}$ in the trapdoor generation phase of session $s(i)$, the trapdoor witness for WI is available to $\mathcal{S}$.

Again, observe that $\mathcal{S}$ finishes the super-polynomial time extraction before the start of secure computation phase in session $s(i)$. Thus, by the non-uniform witness-indistinguishability property of WI, we have

$$H_{i:4} \stackrel{c}{\approx} H_{i:3} \quad (17)$$

Furthermore, by a similar proof to that for [Eq. \(10\)](#), we have

$$\forall \ell \in [m], v_{i:4}^\ell \stackrel{c}{\approx} v_{i:3}^\ell \quad (18)$$

Remark 3. More formally, we can only change the usage of trapdoor witness in WI one instance at a time, but by a standard hybrid argument, we can extend this proof to multiple WI cases.

- Hybrid $H_{i:5}$: This is identical to $H_{i:4}$ except if the i_{th} special commitment corresponds to $\mathbf{c}_{\mathcal{A}}^{\tau, s(i)}$ for session $s(i)$, then $\mathcal{S}$ commits to 0_s instead of $x_H || r_H$ in $\mathbf{c}_H^{s(i)}$ in the input commitment phase of session $s(i)$. Note that $\mathcal{S}$ finishes the super-polynomial time extraction before the start of the input commitment phase of session $s(i)$. Thus, by the non-uniform hiding property of Com , we have

$$H_{i:5} \stackrel{c}{\approx} H_{i:4} \quad (19)$$

Additionally, by the same proof for [Eq. \(10\)](#), we have

$$\forall \ell \in [m], v_{i:5}^\ell \stackrel{c}{\approx} v_{i:4}^\ell \quad (20)$$

- Hybrid $H_{i:6}$: This is identical to $H_{i:5}$ except if the i_{th} special commitment corresponds to $\mathbf{c}_{\mathcal{A}}^{s(i)}$ (in the Input Commitment Phase) for session $s(i)$, then $\mathcal{S}$ follows the steps in [Phase 4.1](#) of the final simulator (except for [Step 2b](#), which is done in $H_{i:4}$).

Claim 5.

$$H_{i:6} \stackrel{c}{\approx} H_{i:5} \quad (21)$$

$$\forall \ell \in [m], v_{i:6}^\ell \stackrel{c}{\approx} v_{i:5}^\ell \quad (22)$$

Proof. In order to show [Eq. \(21\)](#), for the sake of contradiction, assume that there exists a non-uniform QPT distinguisher $\mathcal{D}$, such that it distinguishes $H_{i:6}$ from $H_{i:5}$ with non-negligible probability, then we build in [Algo. 4.3](#), a non-uniform QPT adversary $\mathcal{A}^*$ for the semi-honest security of $\langle P_A^{\text{sh}}, P_B^{\text{sh}} \rangle$. Moreover, we assume that in this specific session $s(i)$, P_B is corrupted. $\mathcal{A}^*$'s behavior is symmetric in sessions where P_A is corrupted instead. We present the simulator in [Algo. 4.1](#).

Algorithm 4.3: $\mathcal{A}$ against semi-honest security of $\langle P_A^{\text{sh}}, P_B^{\text{sh}} \rangle$

$\mathcal{A}^*$ internally emulates the algorithm $\mathcal{D}_\lambda^*(\rho_\lambda)$ (dubbed $\mathcal{D}$ henceforth), while externally interacting with a challenger of the semi-honest security of $\langle P_A^{\text{sh}}, P_B^{\text{sh}} \rangle$. In particular, $\mathcal{A}^*$ proceeds as follows.

1. $\mathcal{A}^*$ acts identical to $H_{i:5}$ until the beginning of [Phase 4.1](#) of the final simulator.
2. $\mathcal{A}^*$ queries the ideal functionality $\mathcal{F}$ using $x_B^{s(i)}$ (extracted in previous hybrids) together with sid $s(i)$, and receives $y^{s(i)}$ from $\mathcal{F}$.
3. $\mathcal{A}^*$ sends $s(i), y^{s(i)}, x_A^{s(i)}, x_B^{s(i)}$ to the external challenger, where $x_A^{s(i)}$ is the honest party's input in session $s(i)$.
4. $\mathcal{A}^*$ receives a transcript for $\langle P_A^{\text{sh}}, P_B^{\text{sh}} \rangle$ and randomness r_A from the challenger.
5. $\mathcal{A}^*$ forces this transcript and randomness on P_B (follows the steps in [Phase 4.1](#) of the final simulator except for [Step 2b](#))
6. $\mathcal{A}^*$ finishes the rest of simulation (with $\mathcal{A}_{\Pi(\mathcal{F})}$) identical to $H_{i:5}$
7. $\mathcal{A}^*$ sends its transcript with $\mathcal{A}_{\Pi(\mathcal{F})}$ to internal distinguisher $\mathcal{D}$, and outputs its decision bit.

Clearly, $\mathcal{A}^*$ runs in polynomial time, and notice that

- if external challenger computes the transcript and output using $x_A^{s(i)}$ and $x_B^{s(i)}$, by all previous hybrids, the invariant condition still holds at this point, then $\mathcal{A}^*$'s transcript with $\mathcal{A}_{\Pi(\mathcal{F})}$ is identical to the one generated in $H_{i:5}$, from $\mathcal{D}$'s perspective.
- if external challenger computes the transcript and output using $y^{s(i)}$ and $x_B^{s(i)}$, then $\mathcal{A}^*$'s transcript with $\mathcal{A}_{\Pi(\mathcal{F})}$ is identical to the one generated in $H_{i:6}$, from $\mathcal{D}$'s perspective.

Therefore, by the assumption that $\mathcal{D}$ distinguishes $H_{i:6}$ from $H_{i:5}$ with non-negligible probability, $\mathcal{A}^*$ breaks the non-uniform semi-honest security of $\langle P_A^{\text{sh}}, P_B^{\text{sh}} \rangle$, and this leads to a contradiction. Therefore, $H_{i:6} \stackrel{c}{\approx} H_{i:5}$.

The proof for [Eq. \(22\)](#) is similar to the proof for [Eq. \(10\)](#), $\forall \ell \in [m]$, we consider the following two cases:

1. if the first message of $\mathbf{c}_A^{\text{NM}, \ell}$ of session ℓ is sent before the start of the **Phase III** of session $s(i)$, then by the first-message-binding property (defined in [Def. 12](#)) of NMCOM, we have $v_{i:6}^\ell = v_{i:5}^\ell$.
2. if the first message of $\mathbf{c}_A^{\text{NM}, \ell}$ of session ℓ is sent after (or simultaneously) the start of the **Phase III** of session $s(i)$. The total number of messages sent in **Phase III** of session $s(i)$ is $2 * t * (1 + \text{rc}(\text{WI})) + 2$, then by the k -robust property of NMCOM (recall that we set $k \geq 2 * t * (1 + \text{rc}(\text{WI})) + 2$), we have $v_{i:6}^\ell \stackrel{c}{\approx} v_{i:5}^\ell$

□

Combining all previous hybrids, it is obvious that $H_{2m+1:0} (H_1)$ is indistinguishable from H_0 and therefore finishes the proof of security.

□

5 Two-Round SPS-ZK for $\text{NP} \cap \text{coNP}$

In this section, we construct a two-round SPS-ZK protocol for $\text{NP} \cap \text{coNP}$ from polynomial hardness assumptions.

5.1 Building Blocks

5.1.1 ZAPs for Controlled-OR Languages

Definition 18 (Controlled-OR Languages). Let $\mathcal{L}_c$ be a language in $\text{NP} \cap \text{coNP}$. Let $\mathcal{L}$ be a language in NP . We call the following language $\tilde{\mathcal{L}}$ the $\mathcal{L}_c$ -Controlled-OR language

$$\tilde{\mathcal{L}} = \mathcal{L}_c \vee \mathcal{L} := \{(x_c, x) \mid x_c \in \mathcal{L}_c \vee x \in \mathcal{L}\}.$$

For the languages define in [Def. 18](#), it is straightforward to see that $\tilde{L} \in \mathbf{NP}$. It is also worth noting that $\tilde{L}$ may not be in $\mathbf{coNP}$, because $\mathcal{L}$ is not necessarily contained in $\mathbf{coNP}$.

Definition 19 (ZAPs for Controlled-OR Languages). *Let $\mathcal{L}_c \in \mathbf{NP} \cap \mathbf{coNP}$, $\mathcal{L} \in \mathbf{NP}$, and $\tilde{\mathcal{L}} \in \mathbf{NP}$ be as in [Def. 18](#). Let $\bar{\mathcal{L}}_c$ be the complement of $\mathcal{L}_c$. A ZAP for the $\mathcal{L}_c$ -controlled language $\tilde{\mathcal{L}}$ consists of the following three algorithms:*

- $V_1(1^\lambda, 1^{|x_c|})$: *This is a PPT algorithm that on input the security parameter 1^λ and an $x_c \in \mathcal{L}_c$, outputs the first ZAP message $\mathbf{zap}_1$.*
- $P((x_c, x), \tilde{w}, \mathbf{zap}_1)$: *This is a PPT algorithm that on input a statement pair (x_c, x) , a witness $\tilde{w}$ for the fact that $(x_c, x) \in \tilde{\mathcal{L}}$, and the first ZAP message $\mathbf{zap}_1$, outputs the second ZAP message $\mathbf{zap}_2$.*
- $V_2((x_c, x), \mathbf{zap}_1, \mathbf{zap}_2)$: *This is a deterministic polynomial-time algorithm that on input a statement pair (x_c, x) and the two ZAP messages $\mathbf{zap}_1$ and $\mathbf{zap}_2$, and outputs a decision symbol $d \in \{\top, \perp\}$, indicating acceptance (when $d = \top$) or rejection (when $d = \perp$).*

The following properties are satisfied:

1. Completeness: For all $x_c \in \mathcal{L}_c$, all x , and all witness $\tilde{w}$ for $(x_c, x) \in \tilde{\mathcal{L}}$, it holds that

$$\Pr[\mathbf{zap}_1 \leftarrow V_1(1^\lambda, x_c); \mathbf{zap}_2 \leftarrow P((x_c, x), \tilde{w}, \mathbf{zap}_1) : V_2((x_c, x), \mathbf{zap}_1, \mathbf{zap}_2) = \top] = 1 - \text{negl}(\lambda).$$

Note that we do not require that $x \in \mathcal{L}$. But as long as $x_c \in \mathcal{L}_c$, it must hold that $(x_c, x) \in \tilde{\mathcal{L}}$ (and thus the $\tilde{w}$ specified above must exist).

2. NO-Controlled Adaptive Soundness: For all $x_c \in \bar{\mathcal{L}}_c$ and all PPT adversary P^* , it holds that

$$\Pr \left[\begin{array}{l} \mathbf{zap}_1 \leftarrow V_1(1^\lambda, x_c); \\ (x^*, \mathbf{zap}_2) \leftarrow P^*(x_c, \mathbf{zap}_1) \end{array} : (x^* \notin \mathcal{L}) \wedge (V_2((x_c, x^*), \mathbf{zap}_1, \mathbf{zap}_2) = \top) \right] \leq \text{negl}(\lambda).$$

3. YES-Controlled Statistical WI: For all $x_c \in \mathcal{L}_c$ and all (potentially unbounded) adversarial verifier V^* , it holds that

$$\left| \Pr[\text{Expr}_{\mathcal{L}_c, \mathcal{L}}^{\text{WI}}(V^*, x_c) = 1] - \frac{1}{2} \right| \leq \text{negl}(\lambda),$$

where $\text{Expr}_{\mathcal{L}_c, \mathcal{L}}^{\text{WI}}(V^*, x_c)$ denotes the output of [Expr. 1](#).

Experiment 1: Statistical WI Experiment $\text{Expr}_{\mathcal{L}_c, \mathcal{L}}^{\text{WI}}(V^*, x_c)$
1. V^*, on input x_c, generates and sends $(\mathbf{zap}_1, x^*, \tilde{w}_0, \tilde{w}_1)$. 2. The experiment checks if both $\tilde{w}_0$ and $\tilde{w}_1$ are valid witnesses for the fact that $(x_c, x^*) \in \tilde{\mathcal{L}}$ (recall from Def. 18 that the language $\tilde{\mathcal{L}}$ is defined as $\tilde{\mathcal{L}} := \mathcal{L}_c \vee \mathcal{L}$): – If so, it samples $b \xleftarrow{\\$} \{0, 1\}$, executes $\mathbf{zap}_2 \leftarrow P((x_c, x^*), \mathbf{zap}_1, \tilde{w}_b)$, and sends $\mathbf{zap}_2$ to V^*; – Otherwise, it halts immediately and outputs a random bit. 3. V^* outputs a bit b^*. If $b^* = b$, the experiment outputs 1; otherwise it outputs 0.

As we mentioned earlier, although the definition of ZAPs for super-complement languages in [\[CGH⁺21\]](#) does not directly imply [Def. 18](#), their construction nonetheless satisfies [Def. 18](#), and the corresponding security proof remains essentially unchanged.

Lemma 2 ([\[CGH⁺21, CCLM22\]](#)). *Assuming QLWE, for any controlled-OR language $\tilde{\mathcal{L}} = \mathcal{L}_c \vee \mathcal{L}$ as per [Def. 18](#), there exist ZAPs for $\tilde{\mathcal{L}}$ as per [Def. 19](#).*

5.2 Construction

We present our construction of two-round SPS-ZK for $\mathbf{NP} \cap \mathbf{coNP}$ in [Prot. 3](#), which is based on the following building blocks:

- A collision-resistant hash function (Gen, H) with input length $\ell^{\text{in}}(\lambda)$ and output length $\ell^{\text{out}}(\lambda)$ (as per Def. 4).
- A ZAP for Controlled-OR Languages (as per Def. 19).
- A two-message function-hiding SFE (as per Def. 3).

As mentioned earlier, all these building blocks can be instantiated based on the polynomial-hardness of QLWE.

Protocol 3: Post-Quantum Two-Round SPS ZK for $\text{NP} \cap \text{coNP}$
Parameter: Let $\ell^{\text{out}}(\lambda) := \frac{\lambda}{k}$ for some constant $k > 2$. Input: For any $\mathcal{L} \in \text{NP} \cap \text{coNP}$, $x \in \mathcal{L} \cap \{0, 1\}^\lambda$, and $w \in \mathcal{R}_{\mathcal{L}}(x)$ denote P's private input. – Round 1: V proceeds as follows: 1. V samples a SFE key $k \leftarrow \text{SFE.Gen}(1^\lambda)$ and generates $\text{ct} \leftarrow \text{SFE.Enc}(k, 0^{\ell(x)})$, where $\ell(x)$ denotes the length of no-witness of the language^a. 2. V samples a hash key $s \leftarrow \text{Gen}(1^\lambda)$; 3. V generates the first ZAP message $\text{zap}_1 \leftarrow V_1(1^\lambda, x)$. V sends $(\text{ct}, s, \text{zap}_1)$. – Round 2: P proceeds as follows: 1. P generates $\hat{\text{ct}} \leftarrow \text{Eval}(C^0, \text{ct})$, where C^0 is a constant-0 circuit that always output 0s. 2. Additionally, P computes $\text{zap}_2 \leftarrow P((x, \hat{\text{ct}}), \tilde{w}, \text{zap}_1)$ proving that either $x \in \mathcal{L}$, or $H^s(x_1) = H^s(x_2)$ and $\hat{\text{ct}}$ is an output of $\text{Eval}(C^{x_1, x_2}, \text{ct})$, where the description of C^{x_1, x_2} is given in Algo. 5.1. Formally, zap_2 proves that $(x, \hat{\text{ct}}) \in \mathcal{L} \vee \mathcal{L}_{\text{ct}}$, and $\mathcal{L} \vee \mathcal{L}_{\text{ct}}$ denotes the controlled-OR Language (also as $\mathcal{L}$-Controlled-OR language), and $\mathcal{L}_{\text{ct}} := \left\{ \hat{\text{ct}} \mid \exists x_1, x_2 \text{ s.t. } \wedge \begin{matrix} \hat{\text{ct}} = \text{Eval}(C^{x_1, x_2}, \text{ct}) \\ H^s(x_1) = H^s(x_2) \end{matrix} \right\}. \quad (23)$ P sends $(\hat{\text{ct}}, \text{zap}_2)$. V's Decision: V outputs the value of $V_2((x, \hat{\text{ct}}), \text{zap}_1, \text{zap}_2)$.
^a (W.l.o.g., we assume that for any given language, the length of its witness/no-witness is some fixed, public polynomial on the length of the instance.)

Algorithm 5.1: Description of C^{x_1, x_2}
Input: Inp Hardware: instance $x, x_1, x_2 \in \{0, 1\}^{\ell^{\text{in}}(\lambda)}$ – Check if $\text{Inp} \in \mathcal{R}_{\mathcal{L}}(x)$ • If true: outputs x_1, x_2; • Otherwise: outputs 0s.

Remark 4. Prot. 3 can in fact be made public-coin, provided that the first message containing the ciphertext ct of the SFE protocol is itself public-coin. Such an SFE instantiation is known under specific LWE-based constructions, as shown in [BD18].

5.3 Proof of Security

Theorem 6. Prot. 3 is a super-polynomial time simulatable zero-knowledge argument.

Completeness and round complexity are straightforward from the description of Prot. 3, and the rest of the proof of Thm. 6 is given in Sec. 5.4 and 5.5.

5.4 Proof of Soundness

Lemma 3. *The protocol described in [Prot. 3](#) is sound against any non-uniform QPT prover $\{P_\lambda^*, \rho_\lambda\}_{\lambda \in \mathbb{N}}$.*

Proof. For the sake of contradiction, assume the existence of a non-uniform QPT prover $\{P_\lambda^*, \rho_\lambda\}_{\lambda \in \mathbb{N}}$ and $x \notin L$, such that $V_2((x, \hat{\text{ct}}), \text{zap}_1, \text{zap}_2) = 1$ with non-negligible probability.

Levering this $\{P_\lambda^*, \rho_\lambda\}_{\lambda \in \mathbb{N}}$, we construct in [Algo. 5.2](#) a (non-uniform) QPT algorithm $\mathcal{A} = \{\mathcal{A}_\lambda, \rho_\lambda\}_{\lambda \in \mathbb{N}}$ breaking the collision-resistant property of (Gen, H) (as per [Def. 4](#)):

Algorithm 5.2: $\mathcal{A}$ against collision-resistance
$\mathcal{A}$ internally emulates the algorithm $\{P_\lambda^*, \rho_\lambda\}_{\lambda \in \mathbb{N}}$ (dubbed P^* henceforth), while externally interacting with a challenger of the collision-resistance of (Gen, H) . In particular, $\mathcal{A}$ proceeds as follows. 1. $\mathcal{A}$ receives a hash key s from the external challenger. 2. $\mathcal{A}$ samples a SFE key $k \leftarrow \text{SFE}$, computes $\text{zap}_1 \leftarrow V_1(1^\lambda, x)$, and $\text{ct} \leftarrow \text{SFE.Enc}(k, \bar{w})$, where $\bar{w} \in \mathcal{R}_{\bar{\mathcal{L}}}(x)$ (by the assumption that $x \notin \mathcal{L}$). 3. $\mathcal{A}$ sends $s, \text{ct}, \text{zap}_1$ to P^*. 4. $\mathcal{A}$ invokes P^* internally to receive $\hat{\text{ct}}, \text{zap}_2$. If $V_2((x, \hat{\text{ct}}), \text{zap}_1, \text{zap}_2) = 1$, then $\mathcal{A}$ outputs $\text{Dec}(k, \hat{\text{ct}})$; Otherwise outputs $\perp$.

Clearly, $\mathcal{A}$ runs in polynomial time, and notice that the hash key s is sampled uniformly by the external challenger. By the post-quantum input privacy of SFE (as per [Property 2](#)), ct is indistinguishable from $\text{SFE.Enc}(k, 0^{\ell(|x|)})$. Therefore, from the P^* 's perspective, this is exactly the same execution as [Prot. 3](#), and thus it will output $\hat{\text{ct}}, \text{zap}_2$, where $V_2((x, \hat{\text{ct}}), \text{zap}_1, \text{zap}_2) = 1$ with non-negligible probability due to our assumption.

Since $x \notin \mathcal{L}$, by the NO-Controlled adaptive soundness of ZAP ([Property 2](#)), $\hat{\text{ct}} \in \mathcal{L}_{\text{ct}}$ with a non-negligible probability. By definition of $\mathcal{L}_{\text{ct}}$, $\text{Dec}(k, \hat{\text{ct}}) = x_1, x_2$, and x_1, x_2 form a collision, with a non-negligible probability. By the fact that (Gen, H) is collision-resistant, this leads to a contradiction. $\square$

5.5 Proof of Zero-Knowledge

Lemma 4. *The protocol described in [Prot. 3](#) is super-polynomial time simulatable zero-knowledge.*

Proof. We first construct a $2^{(2 \cdot \lambda/k)}$ time simulator $\mathcal{S}$. $\mathcal{S}$ works as follow:

- **First Message:** $\mathcal{S}$ receives $s, \text{ct}, \text{zap}_1$ from V^*
- **Second Message:**
 1. $\mathcal{S}$ first runs in $2^{(2 \cdot \lambda/k)}$ ¹⁵ time to brute-force search for a collision x_1 and x_2 , such that $H^s(x_1) = H^s(x_2)$.
 2. $\mathcal{S}$ computes $\hat{\text{ct}} \leftarrow \text{Eval}(C^{x_1, x_2}, \text{ct})$
 3. $\mathcal{S}$ computes zap_2 using x_1, x_2 as the witness.
 4. $\mathcal{S}$ sends $\hat{\text{ct}}, \text{zap}_2$ to the verifier

We now consider the following sequence of hybrids.

- **Hybrid H_0 :** This is the real experiment where P computes $\hat{\text{ct}} \leftarrow \text{Eval}(C^0, \text{ct})$, and use w in ZAP. It outputs the final output of malicious verifier $V_\lambda^*(\rho_\lambda)$.
- **Hybrid H_1 :** This is identical to H_0 except that it runs in $2^{(2 \cdot \lambda/k)}$ time to brute-force search for a collision x_1 and x_2 such that $H^s(x_1) = H^s(x_2)$. If a collision does not exist, it outputs $\perp$, and otherwise works similarly to the previous hybrid. We note that this experiment obtains a collision but does not use it. By

¹⁵ One way to find a collision in time $2^{(2 \cdot \lambda/k)}$ is to check, for each possible image y (out of $2^{\lambda/k}$ values), whether there exist two or more distinct preimages (among the first $2^{\lambda/k} + 1$ inputs) that map to y under the hash function.

the compressing property (Property 1) of (Gen, H) , the probability that this hybrid outputs $\perp$ is 0. Thus, we have

$$H_1 = H_0 \quad (24)$$

- **Hybrid H_2** : This is identical to H_1 except that it computes $\hat{\text{ct}} = \text{Eval}(C^{x_1, x_2}, \text{ct})$ instead of $\text{Eval}(C^0, \text{ct})$. Note that $x \in \mathcal{L}$, no $\bar{w} \in \mathcal{R}_{\bar{\mathcal{L}}}(x)$ exists, therefore C^0 and C^{x_1, x_2} are functionally-equivalent, then by the statistical circuit privacy property of SFE (defined in Property 3), we have

$$H_2 \stackrel{s}{\approx} H_1 \quad (25)$$

- **Hybrid H_3** : This is identical to H_2 except that it uses the fact that $\hat{\text{ct}} = \text{Eval}(C^{x_1, x_2}, \text{ct}) \wedge H^s(x_1) = H^s(x_2)$, as the witness to compute zap_2 . By the YES-Controlled Statistical WI property of ZAP (Property 3), we have

$$H_3 \stackrel{s}{\approx} H_2 \quad (26)$$

Note that H_3 is equivalent to the simulated output of $\mathcal{S}$, therefore combining Eq. (24) to (26), we obtain Lem. 4. $\square$

6 Two-Round SPS-ZK for NP

In this section, we construct a two-round SPS-ZK protocol for **NP** from super-polynomial hardness assumptions.

6.1 Non-interactive Super-Polynomial Time Extractable Commitment

Before we describe our two-round super-polynomial time simulatable zero-knowledge argument scheme, we first recall the definition and construction of non-interactive SPS-extractable commitment scheme from [Pas03].

Definition 20. *A non-interactive perfectly-binding commitment scheme ECom is $T(\lambda)$ -extractable if there exists a probabilistic extractor machine $\mathcal{E}$, with running time bounded by $T(\lambda)$, such that for all messages $c = \text{ECom}(x; r)$, $\mathcal{E}(c) = (x, r)$ with overwhelming probability.*

6.1.1 Construction of Non-Interactive SPS-Extractable Commitment

We recall the construction of Pass's $\lambda^{\log^k \lambda}$ -extractable commitment scheme here. Let f be a post-quantum injective one-way function of input length $\ell_{\text{in}}(\lambda)$ with sub-exponential security, and $b : \{0, 1\}^{\ell_{\text{in}}(\lambda)} \rightarrow \{0, 1\}$ be a hardcore predicate for f , i.e. there exists a κ s.t. b is a hardcore for any probabilistic non-uniform quantum adversary with circuit size bounded by $2^{\ell_{\text{in}}(\lambda)^\kappa}$.

Protocol 4: Post-quantum Non-Interactive SPS-Extractable Commitment
Parameter: Let $k := 1 + \frac{1}{\kappa}$ and $\ell_{\text{in}}(\lambda) = \log^k(\lambda)$ Commit Stage: – To commit to bit $v \in \{0, 1\}$, Committer C samples $x \leftarrow \{0, 1\}^{\log^k(\lambda)}$, and sends $(f(x), b(x) \oplus v)$ to the Verifier V. Reveal Stage: – C sends v and x to V; – V accepts if $f(x) = y$ and $b(x) \oplus v = \beta$ where (α, β) is the V's view of the commit stage.

Lemma 5 ([Pas03, Proposition 3]). *Prot. 4 is $\lambda^{O(\log^k(\lambda))}$ -extractable bit-commitment scheme*

6.2 Post-Quantum Two-Round Super-Polynomial Time Simulatable Zero-Knowledge

We construct a post-quantum two-round super-polynomial time simulatable zero-knowledge argument system Π . Its description is given in [Prot. 5](#). This construction is based on the following building blocks:

- A non-interactive $\lambda^{\log^{\kappa'}} \lambda$ -extractable commitment scheme ECom (as per [Def. 20](#)).
- A sub-exponential $2^{-\ell^{\text{out}}(\lambda)^\kappa}$ secure collision-resistant hash function (Gen, H) with input length $\ell^{\text{in}}(\lambda)$ and output length $\ell^{\text{out}}(\lambda)$ (as per [Def. 5](#)).
- A two-round witness indistinguishable argument scheme (V, P, Verify) for NP (as per [Def. 16](#))¹⁶.

Protocol 5: Post-quantum Two-Round SPS-ZK for NP

Parameter: Let $k = \frac{1}{\kappa} + \frac{2\kappa'}{\kappa}$, $\ell^{\text{out}}(\lambda) = \log^k(\lambda)$ and $\ell^{\text{in}}(\lambda) > \ell^{\text{out}}(\lambda)$

Input: Let $x \in \mathcal{L}$ denote the statement, and $w \in R_{\mathcal{L}}(x)$ denote P 's private witness.

Prove Stage:

– **First Message:**

- Verifier computes $s \leftarrow \text{Gen}(1^\lambda)$ and $\text{wi}_1 \leftarrow V(1^\lambda)$, and sends s, wi_1 to the Prover.

– **Second Message:**

1. Prover computes $\text{c}_1 = \text{ECom}(0^{\ell^{\text{in}}(\lambda)} || 0^{\ell^{\text{in}}(\lambda)})$ and $\text{c}_2 = \text{ECom}(w)$;
2. Additionally, Prover computes $\text{wi}_2 \leftarrow P(\text{wi}_1, x, w)$ proving that either c_1 is an ECom of $x_1 || x_2$ such that $H^s(x_1) = H^s(x_2)$, or c_2 is an ECom to w , such that $w \in R_{\mathcal{L}}(x)$.
Formally, wi_2 proves that $(\text{c}_1, \text{c}_2) \in \mathcal{L}_{\mathcal{H}} \vee \mathcal{L}_x$, and $\mathcal{L}_{\mathcal{H}} \vee \mathcal{L}_x$ denotes the OR-composed language, where

$$\mathcal{L}_{\mathcal{H}} := \left\{ \text{c}_1 \mid \bigwedge \frac{\exists x_1, x_2 \text{ s.t. } \text{c}_1 = \text{ECom}(x_1 || x_2)}{H^s(x_1) = H^s(x_2)} \right\}. \quad (27)$$

and

$$\mathcal{L}_x := \{ \text{c}_2 \mid \exists w \text{ s.t. } \text{c}_2 = \text{ECom}(w) \wedge w \in R_L(x) \}. \quad (28)$$

Note that Prover uses the $(\perp, w)$ as the witness when generating this wi_2 .

3. Prover sends $\text{c}_1, \text{c}_2, \text{wi}_2$ to the verifier.

Verification Phase:

- Verifier accepts if $\text{Verify}(x, \text{c}_1, \text{c}_2, \text{wi}_1, \text{wi}_2) = 1$, and rejects otherwise.

6.3 Proof of Security

Theorem 7. *Assume the existence of sub-exponential secure injective one-way function and sub-exponential secure collision resistant hash function, then [Prot. 5](#) is a super-polynomial time simulatable zero-knowledge argument.*

Completeness and round complexity are straightforward from the description of [Prot. 5](#), and the rest of the proof of [Thm. 7](#) is given in [Sec. 6.4](#) and [6.5](#).

6.4 Proof of Soundness

Lemma 6. *The protocol described in [Prot. 5](#) is sound against any non-uniform QPT prover $\{P_\lambda^*, \rho_\lambda\}_{\lambda \in \mathbb{N}}$.*

Proof. For the sake of contradiction, assume the existence of a non-uniform QPT prover $\{P_\lambda^*, \rho_\lambda\}_{\lambda \in \mathbb{N}}$ and $x \notin L$, such that $\text{Verify}(x, \text{c}_1, \text{c}_2, \text{wi}_1, \text{wi}_2) = 1$ with non-negligible probability.

¹⁶ Two-round PQ-WIAs are known from the quasi-polynomial hardness of the learning with errors (LWE) assumption [[BGI⁺17](#), [BD18](#), [LVW19](#), [BFJ⁺20](#), [GJJM20](#), [BLN⁺24](#)].

Since $\text{Verify}(x, c_1, c_2, w_1, w_2) = 1$ with a non-negligible probability, by the soundness of (V, P, Verify) , $(c_1, c_2) \in \mathcal{L}_H \vee \mathcal{L}_x$ with a non-negligible probability. Additionally, by the assumption of $x \notin L$, no $w \in R_L(x)$ exists, therefore $c_2 \notin \mathcal{L}_x$ and $c_1 \in \mathcal{L}_H$ with non-negligible probability. Since (Gen, H) is collision-resistant against any non-uniform QPT adversary, P^* will not be able to find a collision for H^s , and $c_1 \notin \mathcal{L}_H$ with overwhelming probability. Therefore, this leads to a contradiction. More formally, using a standard reduction, we build a $\lambda^{\log^{k'}(\lambda)}$ time adversary $\mathcal{A}$ for sub-exponential secure collision-resistant hash function. $\mathcal{A}$ acts as a receiver when interacting with P^* , it also receives s from an external challenger and forwards it to P^* . Additionally, $\mathcal{A}$ also computes and sends w_1 to P^* . If P^* sends a valid w_2 (along with c_1, c_2) such that $\text{Verify}(x, c_1, c_2, w_1, w_2) = 1$, then $\mathcal{A}$ runs in $\lambda^{\log^{k'}(\lambda)}$ time to extract $x_1 || x_2$ from c_1 . Since $c_2 \notin \mathcal{L}_x$ and $c_1 \in \mathcal{L}_H$ with a non-negligible probability, x_1 and x_2 form a collision with a non-negligible probability. By the fact that (Gen, H) is collision-resistant against size $2^{(\ell^{\text{out}}(\lambda))^\kappa} = 2^{(\log^k(\lambda))^\kappa} = \lambda^{\log^{2k'}(\lambda)} > \lambda^{O(\log^{k'}(\lambda))}$ non-uniform quantum adversary, this leads to a contradiction. $\square$

6.5 Proof of Zero-Knowledge

Lemma 7. *The protocol described in [Prot. 5](#) is super-polynomial time simulatable zero-knowledge.*

Proof. We first construct a $2^{2 \cdot \log^k(\lambda)}$ (bounded by $\lambda^{2 \cdot \log^k(\lambda)}$) simulator $\mathcal{S}$. $\mathcal{S}$ works as follow:

- **First Message:** $\mathcal{S}$ receives s and w_1 from V^*
- **Second Message:**
 1. $\mathcal{S}$ first runs in $O(2^{2 \cdot \log^k(\lambda)})^{17}$ time to brute-force search for a collision x_1 and x_2 such that $H^s(x_1) = H^s(x_2)$.
 2. $\mathcal{S}$ computes $c_1 = \text{ECom}(x_1 || x_2)$ and $c_2 = \text{ECom}(0^m)$, where $m = |w|$
 3. $\mathcal{S}$ computes w_2 using x_1, x_2 as the witness.
 4. $\mathcal{S}$ sends c_1, c_2, w_2 to the verifier

We now consider the following sequence of hybrids.

- **Hybrid H_0 :** This is the real experiment where P commits to $0 || 0$ in c_1 and w in c_2 . It outputs the final output of malicious verifier $V_\lambda^*(\rho_\lambda)$.
- **Hybrid H_1 :** This is identical to H_0 except that it runs in $2^{2 \cdot \log^k(\lambda)}$ time to brute-force search for a collision x_1 and x_2 such that $H^s(x_1) = H^s(x_2)$. If a collision does not exist, it outputs $\perp$, and otherwise works similarly to the previous hybrid. We note that this experiment obtains a collision but does not use it. By the compressing property ([Property 1](#)) of (Gen, H) , the probability that this hybrid outputs $\perp$ is negligible. Thus, we have

$$H_1 \stackrel{s}{\approx} H_0 \tag{29}$$

- **Hybrid H_2 :** This is identical to H_1 except that it commits to $x_1 || x_2$ in c_1 instead of $0 || 0$. By the non-uniform hiding property of ECom , we have

$$H_2 \stackrel{c}{\approx} H_1 \tag{30}$$

- **Hybrid H_3 :** This is identical to H_2 except that it uses the fact that a collision (x_1, x_2) is committed in c_1 as the witness to compute w_2 . By the non-uniform witness indistinguishable property of (V, P, Verify) , we have

$$H_3 \stackrel{s}{\approx} H_2 \tag{31}$$

¹⁷ One way to find a collision in $2^{2 \cdot \log^k(\lambda)}$ time, is check for every image y (out of $2^{\log^k(\lambda)}$), if there are two or more preimages (out of the first $2^{\log^k(\lambda)} + 1$) that have hash value y .

- **Hybrid H_4** : This is identical to H_3 except that it commit to 0^m in c_2 instead of w . By the non-uniform hiding property of **ECom**, we have

$$H_4 \stackrel{c}{\approx} H_3 \tag{32}$$

Note that H_4 is equivalent to the simulated output of $\mathcal{S}$, therefore combining [Eq. \(29\)](#) to [\(32\)](#), we obtain [Lem. 7](#). $\square$

References

- ABF⁺23. Behzad Abdolmaleki, Saikrishna Badrinarayanan, Rex Fernando, Giulio Malavolta, Ahmadreza Rahimi, and Amit Sahai. Two-round concurrent 2PC from sub-exponential LWE. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part I*, volume 14438 of *Lecture Notes in Computer Science*, pages 71–105, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore. [1](#), [2](#)
- ABG⁺21. Amit Agarwal, James Bartusek, Vipul Goyal, Dakshita Khurana, and Giulio Malavolta. Post-quantum multi-party computation. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 435–464, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland. [1](#)
- ACC⁺21. Bar Alon, Hao Chung, Kai-Min Chung, Mi-Ying Huang, Yi Lee, and Yu-Ching Shen. Round efficient secure multiparty quantum computation with identifiable abort. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part I*, volume 12825 of *Lecture Notes in Computer Science*, pages 436–466, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland. [2](#)
- ACL21. Prabhanjan Ananth, Kai-Min Chung, and Rolando L. La Placa. On the concurrent composition of quantum zero-knowledge. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part I*, volume 12825 of *Lecture Notes in Computer Science*, pages 346–374, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland. [1](#), [2](#)
- BCKM21a. James Bartusek, Andrea Coladangelo, Dakshita Khurana, and Fermi Ma. On the round complexity of secure quantum computation. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part I*, volume 12825 of *Lecture Notes in Computer Science*, pages 406–435, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland. [2](#), [3](#)
- BCKM21b. James Bartusek, Andrea Coladangelo, Dakshita Khurana, and Fermi Ma. One-way functions imply secure computation in a quantum world. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part I*, volume 12825 of *Lecture Notes in Computer Science*, pages 467–496, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland. [2](#)
- BD18. Zvika Brakerski and Nico Döttling. Two-message statistically sender-private OT from LWE. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018: 16th Theory of Cryptography Conference, Part II*, volume 11240 of *Lecture Notes in Computer Science*, pages 370–390, Panaji, India, November 11–14, 2018. Springer, Cham, Switzerland. [1](#), [9](#), [23](#), [26](#)
- BDH⁺17. Brandon Broadnax, Nico Döttling, Gunnar Hartung, Jörn Müller-Quade, and Matthias Nagel. Concurrently composable security with shielded super-polynomial simulators. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part I*, volume 10210 of *Lecture Notes in Computer Science*, pages 351–381, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland. [1](#), [3](#)
- BFJ⁺20. Saikrishna Badrinarayanan, Rex Fernando, Aayush Jain, Dakshita Khurana, and Amit Sahai. Statistical ZAP arguments. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part III*, volume 12107 of *Lecture Notes in Computer Science*, pages 642–667, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland. [26](#)
- BGI⁺17. Saikrishna Badrinarayanan, Sanjam Garg, Yuval Ishai, Amit Sahai, and Akshay Wadia. Two-message witness indistinguishability and secure computation in the plain model from new assumptions. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017, Part III*, volume 10626 of *Lecture Notes in Computer Science*, pages 275–303, Hong Kong, China, December 3–7, 2017. Springer, Cham, Switzerland. [1](#), [26](#)
- BLN⁺24. Eli Bradley, George Lu, Shafik Nassar, Brent Waters, and David J. Wu. A hidden-bits approach to statistical ZAPs from LWE. Cryptology ePrint Archive, Report 2024/1663, 2024. [26](#)
- BLS22. Nir Bitansky, Huijia Lin, and Omri Shmueli. Non-malleable commitments against quantum attacks. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 519–550, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland. [6](#), [11](#), [12](#)
- BS05. Boaz Barak and Amit Sahai. How to play almost any mental game over the net - concurrent composition via super-polynomial simulation. In *46th Annual Symposium on Foundations of Computer Science*, pages 543–552, Pittsburgh, PA, USA, October 23–25, 2005. IEEE Computer Society Press. [1](#)
- BS20. Nir Bitansky and Omri Shmueli. Post-quantum zero knowledge in constant rounds. In Konstantin Makarychev, Yury Makarychev, Madhur Tulsiani, Gautam Kamath, and Julia Chuzhoy, editors, *52nd Annual ACM Symposium on Theory of Computing*, pages 269–279, Chicago, IL, USA, June 22–26, 2020. ACM Press. [1](#), [9](#)
- Can01. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd Annual Symposium on Foundations of Computer Science*, pages 136–145, Las Vegas, NV, USA, October 14–17, 2001. IEEE Computer Society Press. [1](#)

- CCLM22. Rohit Chatterjee, Kai-Min Chung, Xiao Liang, and Giulio Malavolta. A note on the post-quantum security of (ring) signatures. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *PKC 2022: 25th International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 13178 of *Lecture Notes in Computer Science*, pages 407–436, Virtual Event, March 8–11, 2022. Springer, Cham, Switzerland. [7](#), [22](#)
- CCLY22. Nai-Hui Chia, Kai-Min Chung, Xiao Liang, and Takashi Yamakawa. Post-quantum simulatable extraction with minimal assumptions: Black-box and constant-round. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part III*, volume 13509 of *Lecture Notes in Computer Science*, pages 533–563, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland. [1](#), [5](#), [13](#)
- CCY21. Nai-Hui Chia, Kai-Min Chung, and Takashi Yamakawa. A black-box approach to post-quantum zero-knowledge in constant rounds. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part I*, volume 12825 of *Lecture Notes in Computer Science*, pages 315–345, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland. [1](#), [5](#)
- CGH⁺21. Rohit Chatterjee, Sanjam Garg, Mohammad Hajiabadi, Dakshita Khurana, Xiao Liang, Giulio Malavolta, Omkant Pandey, and Sina Shiehian. Compact ring signatures from learning with errors. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part I*, volume 12825 of *Lecture Notes in Computer Science*, pages 282–312, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland. [7](#), [22](#)
- CLP10. Ran Canetti, Huijia Lin, and Rafael Pass. Adaptive hardness and composable security in the plain model from standard assumptions. In *51st Annual Symposium on Foundations of Computer Science*, pages 541–550, Las Vegas, NV, USA, October 23–26, 2010. IEEE Computer Society Press. [1](#), [3](#)
- CLP20. Rohit Chatterjee, Xiao Liang, and Omkant Pandey. Improved black-box constructions of composable secure computation. In Artur Czumaj, Anuj Dawar, and Emanuela Merelli, editors, *ICALP 2020: 47th International Colloquium on Automata, Languages and Programming*, volume 168 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 28:1–28:20, Saarbrücken, Germany, July 8–11, 2020. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik. [3](#)
- CLPY25. Rohit Chatterjee, Xiao Liang, Omkant Pandey, and Takashi Yamakawa. The round complexity of black-box post-quantum secure computation. In *Annual International Cryptology Conference*. Springer, 2025. [1](#), [6](#)
- DFS04. Ivan Damgård, Serge Fehr, and Louis Salvail. Zero-knowledge proofs and string commitments withstanding quantum attacks. In Matthew Franklin, editor, *Advances in Cryptology – CRYPTO 2004*, volume 3152 of *Lecture Notes in Computer Science*, pages 254–272, Santa Barbara, CA, USA, August 15–19, 2004. Springer Berlin Heidelberg, Germany. [1](#)
- DGJ⁺20. Yfke Dulek, Alex B. Grilo, Stacey Jeffery, Christian Majenz, and Christian Schaffner. Secure multi-party quantum computation with a dishonest majority. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part III*, volume 12107 of *Lecture Notes in Computer Science*, pages 729–758, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland. [2](#)
- DL09. Ivan Damgård and Carolin Lunemann. Quantum-secure coin-flipping and applications. In Mitsuru Matsui, editor, *Advances in Cryptology – ASIACRYPT 2009*, volume 5912 of *Lecture Notes in Computer Science*, pages 52–69, Tokyo, Japan, December 6–10, 2009. Springer Berlin Heidelberg, Germany. [1](#), [2](#)
- DNS12. Frédéric Dupuis, Jesper Buus Nielsen, and Louis Salvail. Actively secure two-party evaluation of any quantum operation. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 794–811, Santa Barbara, CA, USA, August 19–23, 2012. Springer Berlin Heidelberg, Germany. [2](#)
- GGJS12. Sanjam Garg, Vipul Goyal, Abhishek Jain, and Amit Sahai. Concurrently secure computation in constant rounds. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 99–116, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany. [ii](#), [1](#), [2](#), [3](#), [4](#), [5](#), [6](#), [14](#), [17](#)
- GJ13. Vipul Goyal and Abhishek Jain. On concurrently secure computation in the multiple ideal query model. In Thomas Johansson and Phong Q. Nguyen, editors, *Advances in Cryptology – EUROCRYPT 2013*, volume 7881 of *Lecture Notes in Computer Science*, pages 684–701, Athens, Greece, May 26–30, 2013. Springer Berlin Heidelberg, Germany. [1](#)
- GJJM20. Vipul Goyal, Abhishek Jain, Zhengzhong Jin, and Giulio Malavolta. Statistical zaps and new oblivious transfer protocols. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part III*, volume 12107 of *Lecture Notes in Computer Science*, pages 668–699, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland. [26](#)
- GKP18. Sanjam Garg, Susumu Kiyoshima, and Omkant Pandey. A new approach to black-box concurrent secure computation. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EURO-*

- CRYPT 2018, Part II*, volume 10821 of *Lecture Notes in Computer Science*, pages 566–599, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland. [1](#), [3](#)
- GLM23. Vipul Goyal, Xiao Liang, and Giulio Malavolta. On concurrent multi-party quantum computation. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 129–161, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland. [1](#), [2](#), [3](#)
- GLP⁺15. Vipul Goyal, Huijia Lin, Omkant Pandey, Rafael Pass, and Amit Sahai. Round-efficient concurrently composable secure computation via a robust extraction lemma. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015: 12th Theory of Cryptography Conference, Part I*, volume 9014 of *Lecture Notes in Computer Science*, pages 260–289, Warsaw, Poland, March 23–25, 2015. Springer Berlin Heidelberg, Germany. [3](#)
- GLSV21. Alex B. Grilo, Huijia Lin, Fang Song, and Vinod Vaikuntanathan. Oblivious transfer is in MiniQCrypt. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 531–561, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland. [2](#)
- GMW87. Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred Aho, editor, *19th Annual ACM Symposium on Theory of Computing*, pages 218–229, New York City, NY, USA, May 25–27, 1987. ACM Press. [1](#)
- GPR16. Vipul Goyal, Omkant Pandey, and Silas Richelson. Textbook non-malleable commitments. In Daniel Wichs and Yishay Mansour, editors, *48th Annual ACM Symposium on Theory of Computing*, pages 1128–1141, Cambridge, MA, USA, June 18–21, 2016. ACM Press. [11](#)
- Gra97. Jeroen Van De Graaf. *Towards a formal definition of security for quantum protocols*. PhD thesis, University of Montreal, Montreal, Canada, 1997. [1](#)
- HSS11. Sean Hallgren, Adam Smith, and Fang Song. Classical cryptographic protocols in a quantum world. In Phillip Rogaway, editor, *Advances in Cryptology – CRYPTO 2011*, volume 6841 of *Lecture Notes in Computer Science*, pages 411–428, Santa Barbara, CA, USA, August 14–18, 2011. Springer Berlin Heidelberg, Germany. [1](#), [2](#)
- Kiy14. Susumu Kiyoshima. Round-efficient black-box construction of composable multi-party computation. In Juan A. Garay and Rosario Gennaro, editors, *Advances in Cryptology – CRYPTO 2014, Part II*, volume 8617 of *Lecture Notes in Computer Science*, pages 351–368, Santa Barbara, CA, USA, August 17–21, 2014. Springer Berlin Heidelberg, Germany. [3](#)
- Lin03. Yehuda Lindell. Bounded-concurrent secure two-party computation without setup assumptions. In *35th Annual ACM Symposium on Theory of Computing*, pages 683–692, San Diego, CA, USA, June 9–11, 2003. ACM Press. [1](#)
- Lin04. Yehuda Lindell. Lower bounds for concurrent self composition. In Moni Naor, editor, *TCC 2004: 1st Theory of Cryptography Conference*, volume 2951 of *Lecture Notes in Computer Science*, pages 203–222, Cambridge, MA, USA, February 19–21, 2004. Springer Berlin Heidelberg, Germany. [1](#)
- LMS22. Alex Lombardi, Fermi Ma, and Nicholas Spooner. Post-quantum zero knowledge, revisited or: How to do quantum rewinding undetectably. In *63rd Annual Symposium on Foundations of Computer Science*, pages 851–859, Denver, CO, USA, October 31 – November 3, 2022. IEEE Computer Society Press. [1](#), [5](#), [13](#)
- LN11. Carolin Lunemann and Jesper Buus Nielsen. Fully simulatable quantum-secure coin-flipping and applications. In Abderrahmane Nitaj and David Pointcheval, editors, *AFRICACRYPT 11: 4th International Conference on Cryptology in Africa*, volume 6737 of *Lecture Notes in Computer Science*, pages 21–40, Dakar, Senegal, July 5–7, 2011. Springer Berlin Heidelberg, Germany. [1](#), [2](#)
- LP09. Huijia Lin and Rafael Pass. Non-malleability amplification. In Michael Mitzenmacher, editor, *41st Annual ACM Symposium on Theory of Computing*, pages 189–198, Bethesda, MD, USA, May 31 – June 2, 2009. ACM Press. [6](#), [11](#)
- LP12. Huijia Lin and Rafael Pass. Black-box constructions of composable protocols without set-up. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 461–478, Santa Barbara, CA, USA, August 19–23, 2012. Springer Berlin Heidelberg, Germany. [3](#)
- LPV08. Huijia Lin, Rafael Pass, and Muthuramakrishnan Venkatasubramanian. Concurrent non-malleable commitments from any one-way function. In Ran Canetti, editor, *TCC 2008: 5th Theory of Cryptography Conference*, volume 4948 of *Lecture Notes in Computer Science*, pages 571–588, San Francisco, CA, USA, March 19–21, 2008. Springer Berlin Heidelberg, Germany. [11](#)
- LPV09. Huijia Lin, Rafael Pass, and Muthuramakrishnan Venkatasubramanian. A unified framework for concurrent security: universal composable security from stand-alone non-malleability. In Michael Mitzenmacher, editor, *41st Annual ACM Symposium on Theory of Computing*, pages 179–188, Bethesda, MD, USA, May 31 – June 2, 2009. ACM Press. [1](#)

- LPY23. Xiao Liang, Omkant Pandey, and Takashi Yamakawa. A new approach to post-quantum non-malleability. In *64th Annual Symposium on Foundations of Computer Science*, pages 568–579, Santa Cruz, CA, USA, November 6–9, 2023. IEEE Computer Society Press. [1](#), [2](#), [6](#), [8](#), [10](#), [12](#), [13](#)
- LVW19. Alex Lombardi, Vinod Vaikuntanathan, and Daniel Wichs. 2-message publicly verifiable WI from (subexponential) LWE. Cryptology ePrint Archive, Report 2019/808, 2019. [26](#)
- MMY06. Tal Malkin, Ryan Moriarty, and Nikolai Yakovenko. Generalized environmental security from number theoretic assumptions. In Shai Halevi and Tal Rabin, editors, *TCC 2006: 3rd Theory of Cryptography Conference*, volume 3876 of *Lecture Notes in Computer Science*, pages 343–359, New York, NY, USA, March 4–7, 2006. Springer Berlin Heidelberg, Germany. [1](#)
- OPP14. Rafail Ostrovsky, Anat Paskin-Cherniavsky, and Beni Paskin-Cherniavsky. Maliciously circuit-private FHE. In Juan A. Garay and Rosario Gennaro, editors, *Advances in Cryptology – CRYPTO 2014, Part I*, volume 8616 of *Lecture Notes in Computer Science*, pages 536–553, Santa Barbara, CA, USA, August 17–21, 2014. Springer Berlin Heidelberg, Germany. [9](#)
- Pas03. Rafael Pass. Simulation in quasi-polynomial time, and its application to protocol composition. In Eli Biham, editor, *Advances in Cryptology – EUROCRYPT 2003*, volume 2656 of *Lecture Notes in Computer Science*, pages 160–176, Warsaw, Poland, May 4–8, 2003. Springer Berlin Heidelberg, Germany. [1](#), [7](#), [25](#)
- PS04. Manoj Prabhakaran and Amit Sahai. New notions of security: Achieving universal composability without trusted setup. In László Babai, editor, *36th Annual ACM Symposium on Theory of Computing*, pages 242–251, Chicago, IL, USA, June 13–16, 2004. ACM Press. [1](#), [3](#)
- PW09. Rafael Pass and Hoeteck Wee. Black-box constructions of two-party protocols from one-way functions. In Omer Reingold, editor, *TCC 2009: 6th Theory of Cryptography Conference*, volume 5444 of *Lecture Notes in Computer Science*, pages 403–418. Springer Berlin Heidelberg, Germany, March 15–17, 2009. [4](#)
- Wat09. John Watrous. Zero-knowledge against quantum attacks. *SIAM Journal on Computing*, 39(1):25–58, 2009. [1](#), [5](#)
- Yao86. Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th Annual Symposium on Foundations of Computer Science*, pages 162–167, Toronto, Ontario, Canada, October 27–29, 1986. IEEE Computer Society Press. [1](#)